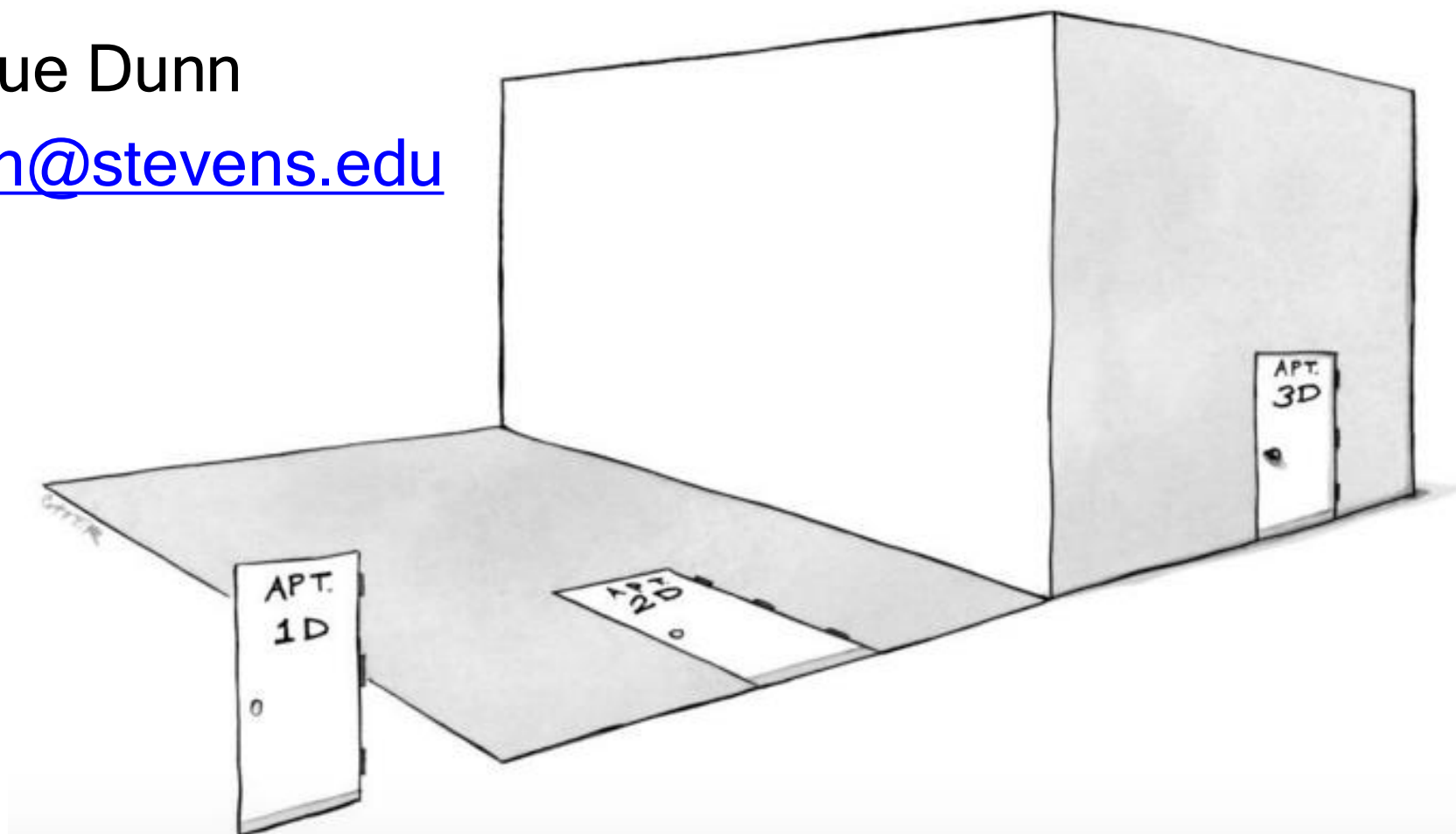


CS 532: 3D Computer Vision

Lecture 4

Enrique Dunn

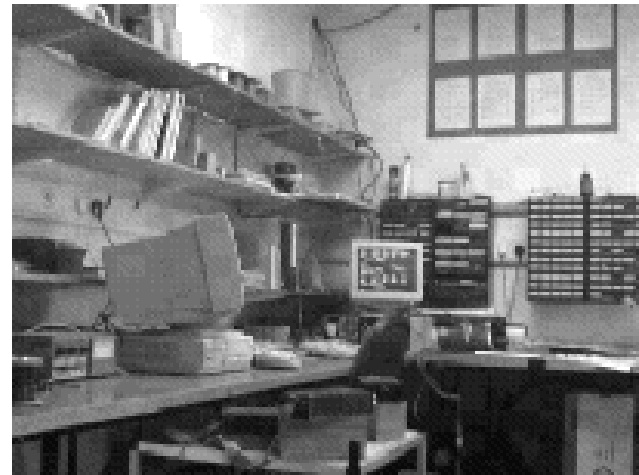
edunn@stevens.edu



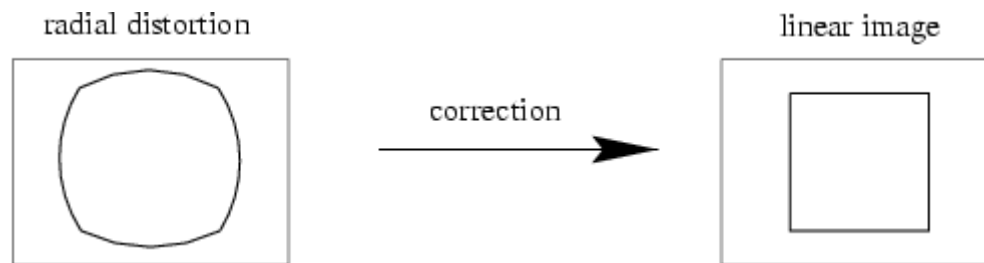
Radial Distortion and Undistortion

Slides by R. Hartley, A. Zisserman
and M. Pollefeys

Radial Distortion



short and long focal length







Typical Undistortion Model

Correction of distortion

$$\hat{x} = x_c + L(r)(x - x_c) \quad \hat{y} = y_c + L(r)(y - y_c)$$

Choice of the distortion function and center

$$\begin{aligned} x &= x_o + (x_o - c_x)(K_1 r^2 + K_2 r^4 + \dots) \\ y &= y_o + (y_o - c_y)(K_1 r^2 + K_2 r^4 + \dots) \end{aligned}$$

$$r = (x_o - c_x)^2 + (y_o - c_y)^2 \ .$$

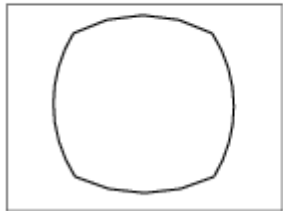
Computing the parameters of the distortion function

- (i) Minimize with additional unknowns
- (ii) Straighten lines
- (iii) ...

Why Undistort?



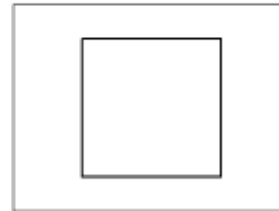
radial distortion



correction



linear image



$$(\tilde{x}, \tilde{y}, 1)^{\top} = [\mathbf{I} \mid \mathbf{0}] \mathbf{X}_{\text{cam}}$$

$$\begin{pmatrix} x_d \\ y_d \end{pmatrix} = L(\tilde{r}) \begin{pmatrix} \tilde{x} \\ \tilde{y} \end{pmatrix}$$

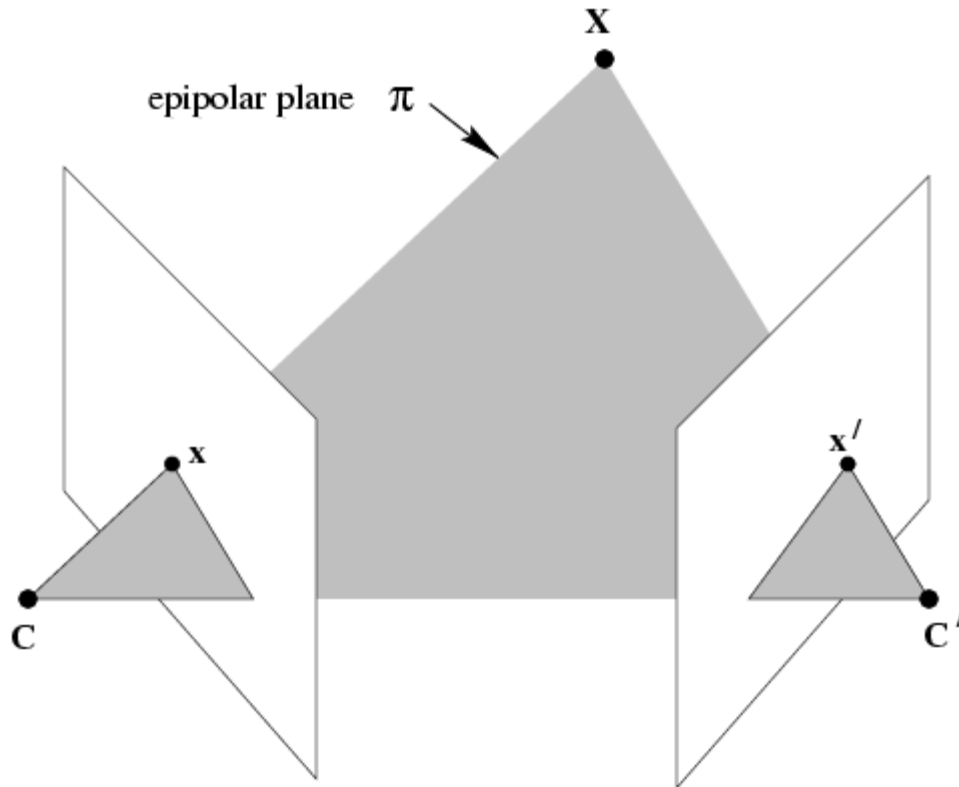
Two-View Geometry

Slides by R. Hartley, A. Zisserman and M. Pollefeys

Three questions:

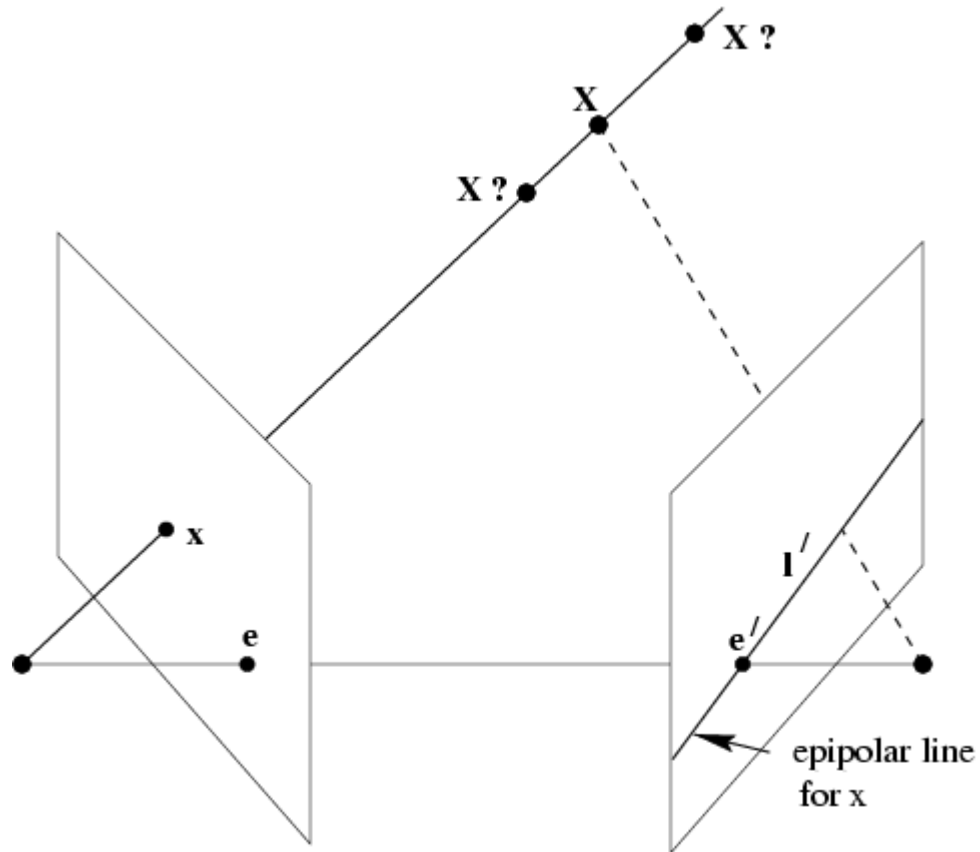
- (i) **Correspondence geometry:** Given an image point X in the first image, how does this constrain the position of the corresponding point X' in the second image?
- (ii) **Camera geometry (motion):** Given a set of corresponding image points $\{x_i \leftrightarrow x'_i\}$, $i=1, \dots, n$, what are the cameras P and P' for the two views?
- (iii) **Scene geometry (structure):** Given corresponding image points $x_i \leftrightarrow x'_i$ and cameras P, P' , what is the position of (their pre-image) X in space?

The Epipolar Geometry



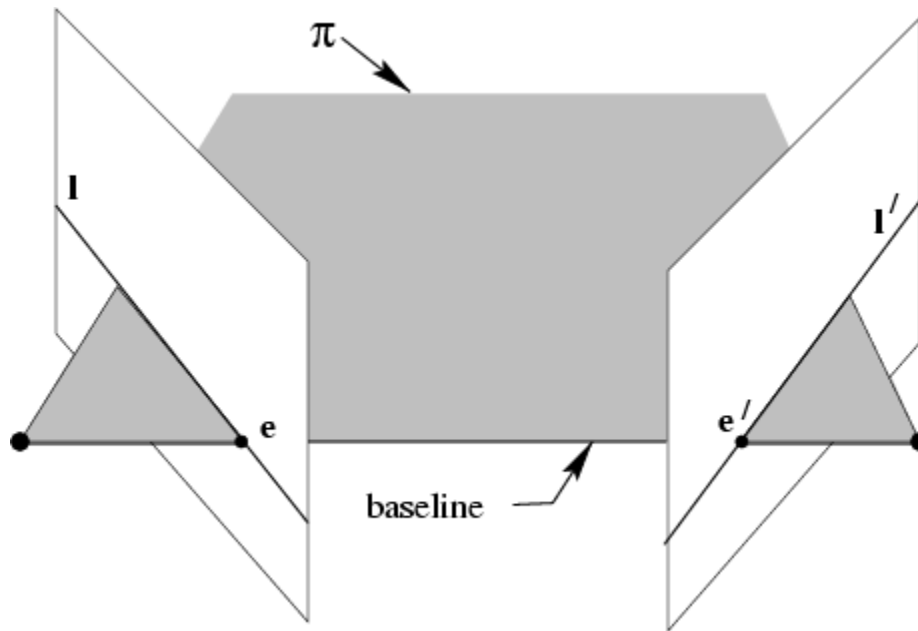
C, C', x, x' and X are coplanar

The Epipolar Geometry



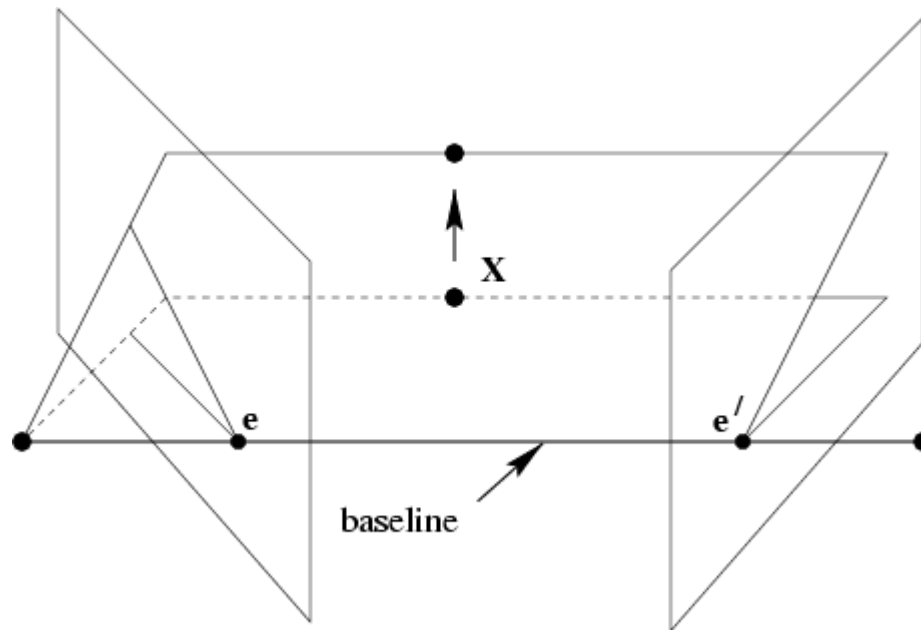
What if only C, C', x are known?

The Epipolar Geometry



All points on π project on l and l'

The Epipolar Geometry



Family of planes π and lines l and l'
Intersection in e and e'

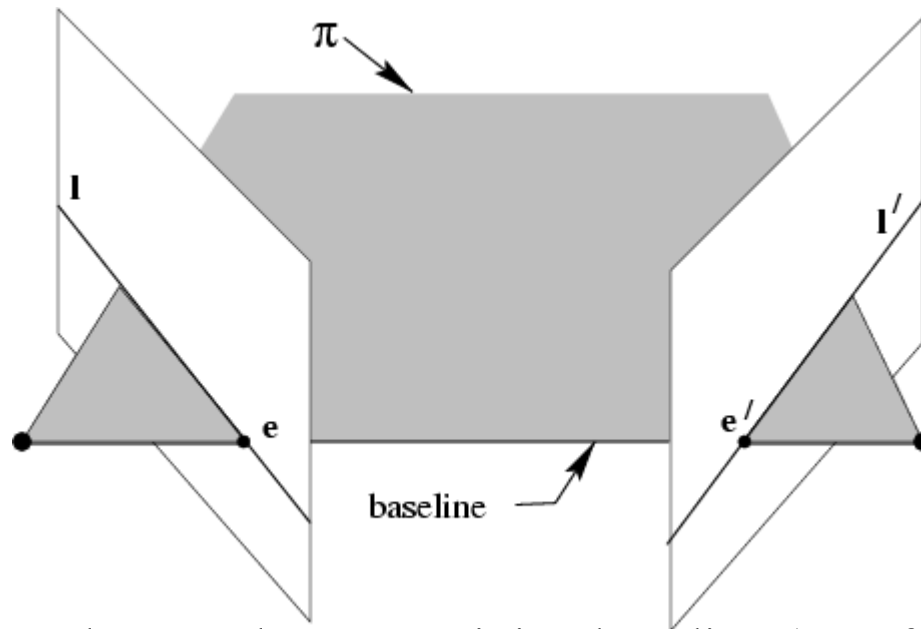
The Epipolar Geometry

epipoles e, e'

= intersection of baseline with image plane

= projection of projection center in other image

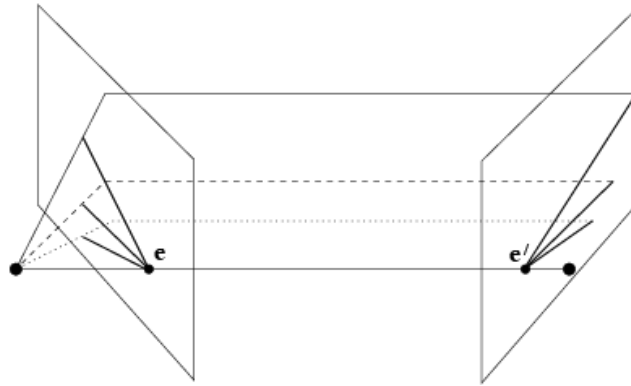
= vanishing point of camera motion direction



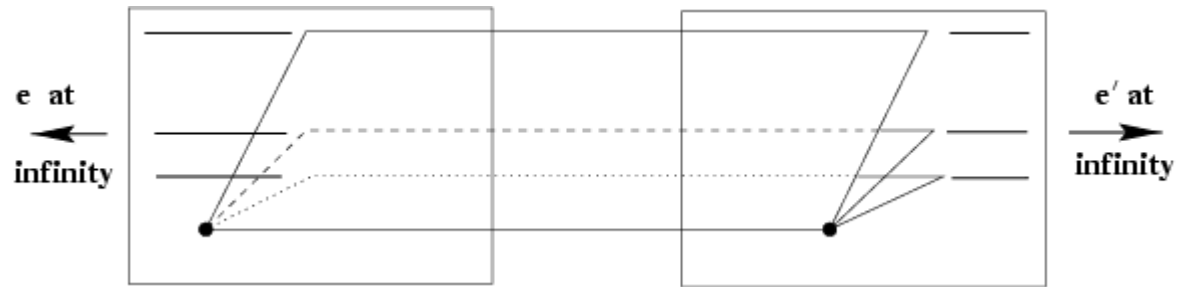
an epipolar plane = plane containing baseline (1-D family)

an epipolar line = intersection of epipolar plane with image
(always come in corresponding pairs)

Example: Converging Cameras

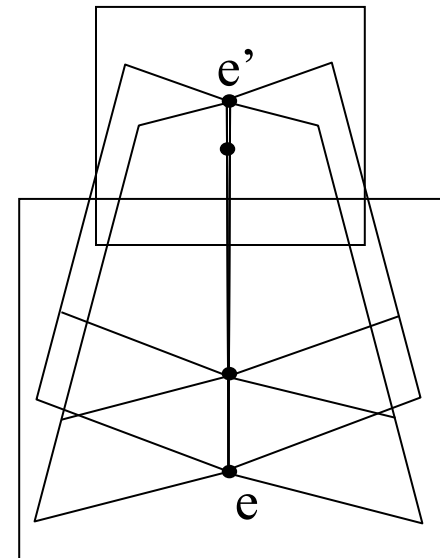
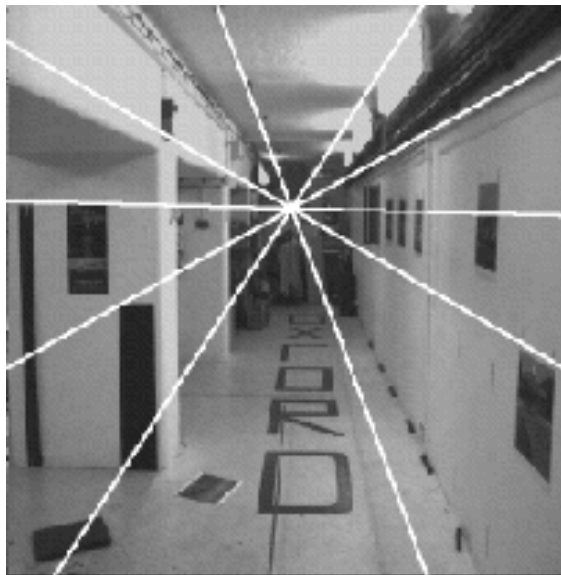
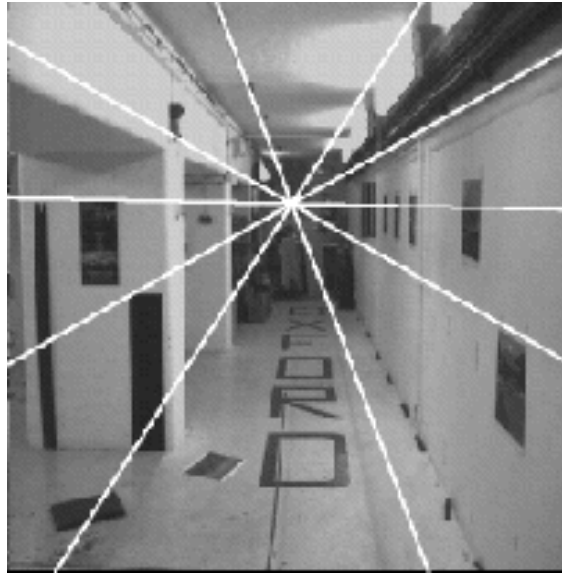


Example: Motion Parallel to Image Plane



(simple for stereo $\rightarrow$ rectification)

Example: Forward Motion



The Fundamental Matrix F

algebraic representation of epipolar geometry

$$\mathbf{x} \mapsto \mathbf{l}'$$

we will see that mapping is a (singular) correlation
(i.e. projective mapping from points to lines)
represented by the fundamental matrix F

The Fundamental Matrix F

correspondence condition

The fundamental matrix satisfies the condition that for any pair of corresponding points $x \leftrightarrow x'$ in the two images

$$x'^T F x = 0 \quad (x'^T l' = 0)$$

The Fundamental Matrix F

F is the unique 3×3 rank 2 matrix that satisfies $x'^T F x = 0$ for all $x \leftrightarrow x'$

- (i) **Transpose:** if F is fundamental matrix for (P,P'), then F^T is fundamental matrix for (P',P)
- (ii) **Epipolar lines:** $l' = Fx$ & $l = F^T x'$
- (iii) **Epipoles:** on all epipolar lines, thus $e'^T F x = 0, \forall x \Rightarrow e'^T F = 0$, similarly $F e = 0$
- (iv) F has 7 d.o.f. , i.e. $3 \times 3 - 1(\text{homogeneous}) - 1(\text{rank } 2)$
- (v) F is a correlation, projective mapping from a point x to a line $l' = Fx$ (not a proper correlation, i.e. not invertible)

Two View Geometry Computation: Linear Algorithm

For every match (m,m'): $\mathbf{x}'^T \mathbf{F} \mathbf{x} = 0$

$$x' x f_{11} + x' y f_{12} + x' f_{13} + y' x f_{21} + y' y f_{22} + y' f_{23} + x f_{31} + y f_{32} + f_{33} = 0$$

separate known from unknown

$$\underbrace{[x' x, x' y, x', y' x, y' y, y', x, y, 1]}_{\text{(data)}} \underbrace{[f_{11}, f_{12}, f_{13}, f_{21}, f_{22}, f_{23}, f_{31}, f_{32}, f_{33}]}_{\substack{\text{(unknowns)} \\ \text{(linear)}}}^T = 0$$

$$\begin{bmatrix} x'_1 x_1 & x'_1 y_1 & x'_1 & y'_1 x_1 & y'_1 y_1 & y'_1 & x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x'_n x_n & x'_n y_n & x'_n & y'_n x_n & y'_n y_n & y'_n & x_n & y_n & 1 \end{bmatrix} \mathbf{f} = 0$$

$$\mathbf{A} \mathbf{f} = 0$$

The Essential Matrix

~fundamental matrix for calibrated cameras (remove K)

$$E = [t]_{\times} R = R[R^T t]_{\times} \quad [a]_{\times} \stackrel{\text{def}}{=} \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}.$$
$$\hat{x}'^T E \hat{x} = 0$$
$$\left(\hat{x} = K^{-1}x; \hat{x}' = K^{-1}x' \right)$$

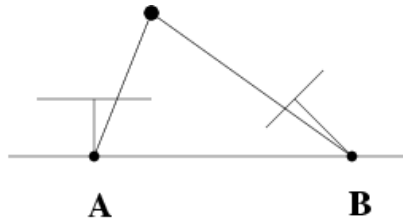
$$E = K'^T F K$$

5 d.o.f. (3 for R; 2 for t up to scale)

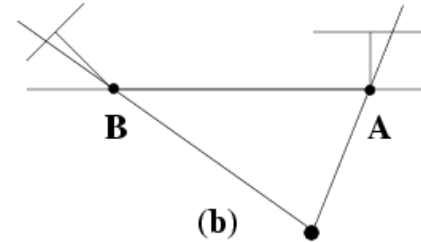
E is an essential matrix if and only if two singular values are equal (and the third=0)

$$E = U \text{diag}(1, 1, 0) V^T$$

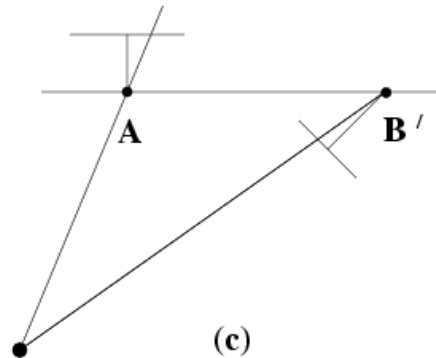
Four Possible Solutions from E



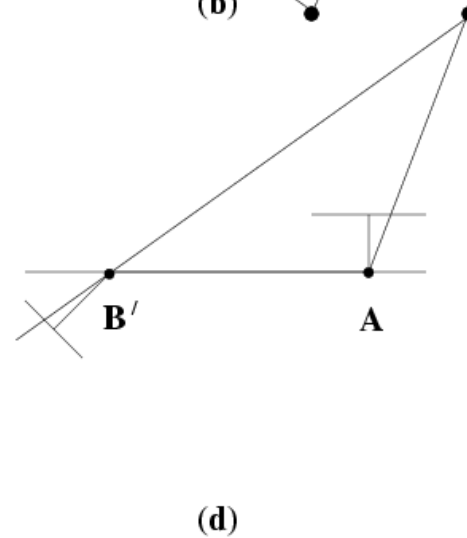
(a)



(b)



(c)



(d)

Given E and setting the first camera matrix $P = [I \mid 0]$, there are four possible solutions for P' (only one solution, however, where a reconstructed point is in front of both cameras)

Fundamental Matrix Estimation

Epipolar Geometry: Basic Equation

$$\mathbf{x}'^T \mathbf{F} \mathbf{x} = 0$$

$$x' x f_{11} + x' y f_{12} + x' f_{13} + y' x f_{21} + y' y f_{22} + y' f_{23} + x f_{31} + y f_{32} + f_{33} = 0$$

separate known from unknown

$$\underbrace{[x'x, x'y, x', y'x, y'y, y', x, y, 1]}_{\text{(data)}} \underbrace{[f_{11}, f_{12}, f_{13}, f_{21}, f_{22}, f_{23}, f_{31}, f_{32}, f_{33}]}_{\substack{\text{(unknowns)} \\ \text{(linear)}}}^T = 0$$

$$\begin{bmatrix} x'_1 x_1 & x'_1 y_1 & x'_1 & y'_1 x_1 & y'_1 y_1 & y'_1 & x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x'_n x_n & x'_n y_n & x'_n & y'_n x_n & y'_n y_n & y'_n & x_n & y_n & 1 \end{bmatrix} \mathbf{f} = 0$$

$$\mathbf{A} \mathbf{f} = 0$$

The Singularity Constraint

$$\mathbf{e}'^T \mathbf{F} = 0 \quad \mathbf{F} \mathbf{e} = 0 \quad \det \mathbf{F} = 0 \quad \text{rank } \mathbf{F} = 2$$

SVD from linearly computed $\mathbf{F}$ matrix (rank 3)

$$\mathbf{F} = \mathbf{U} \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \sigma_3 \end{bmatrix} \mathbf{V}^T = \mathbf{U}_1 \sigma_1 \mathbf{V}_1^T + \mathbf{U}_2 \sigma_2 \mathbf{V}_2^T + \mathbf{U}_3 \sigma_3 \mathbf{V}_3^T$$

Compute closest rank-2 approximation $\min \|\mathbf{F} - \mathbf{F}'\|_F$

$$\mathbf{F}' = \mathbf{U} \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & 0 \end{bmatrix} \mathbf{V}^T = \mathbf{U}_1 \sigma_1 \mathbf{V}_1^T + \mathbf{U}_2 \sigma_2 \mathbf{V}_2^T$$

The Singularity Constraint



The NOT Normalized 8-point Algorithm

$$\begin{bmatrix}
 x_1 x_1' & y_1 x_1' & x_1' & x_1 y_1' & y_1 y_1' & y_1' & x_1 & y_1 & 1 \\
 x_2 x_2' & y_2 x_2' & x_2' & x_2 y_2' & y_2 y_2' & y_2' & x_2 & y_2 & 1 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 x_n x_n' & y_n x_n' & x_n' & x_n y_n' & y_n y_n' & y_n' & x_n & y_n & 1
 \end{bmatrix}
 \begin{bmatrix}
 f_{11} \\
 f_{12} \\
 f_{13} \\
 f_{21} \\
 f_{22} \\
 f_{23} \\
 f_{31} \\
 f_{32} \\
 f_{33}
 \end{bmatrix} = 0$$

~ 10000 ~ 10000 ~ 100 ~ 10000 ~ 10000 ~ 100 ~ 100 ~ 100 1

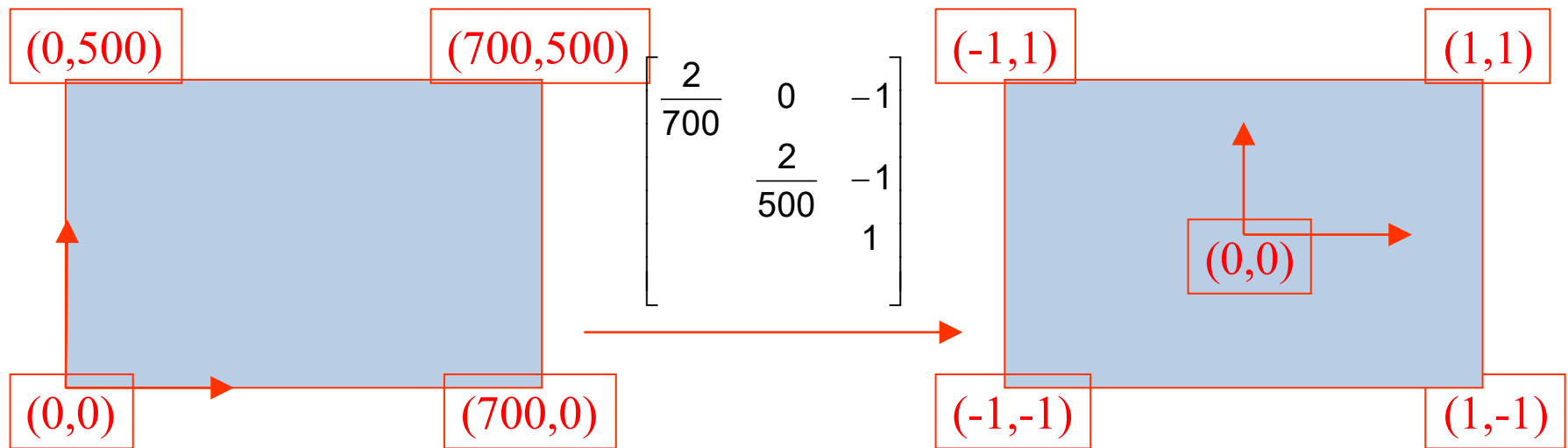


Orders of magnitude difference
 between column of data matrix

→ least-squares yields poor results

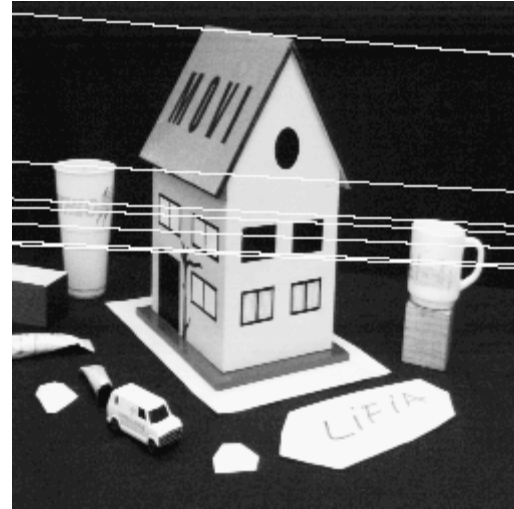
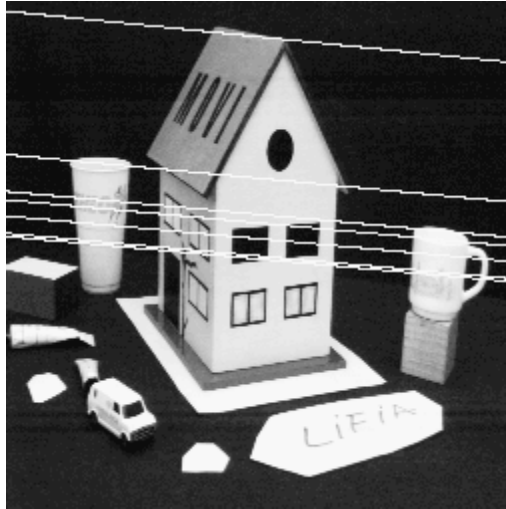
The Normalized 8-point Algorithm

Transform image to $[-1,1] \times [-1,1]$

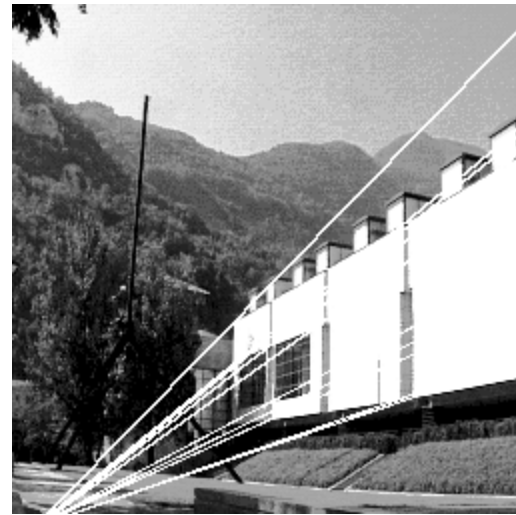
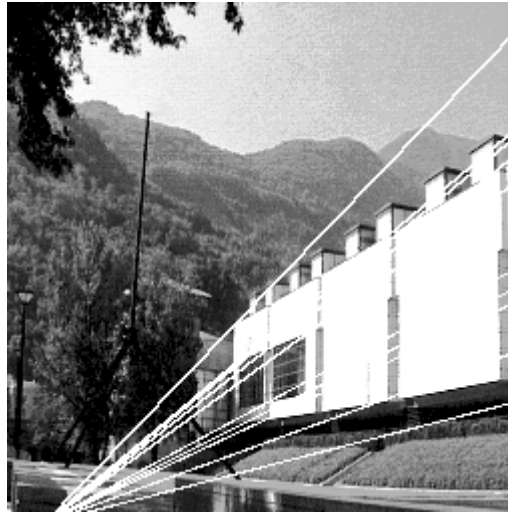


normalized least squares yields good results

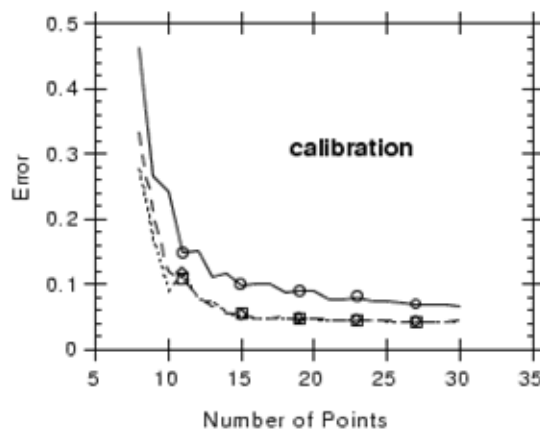
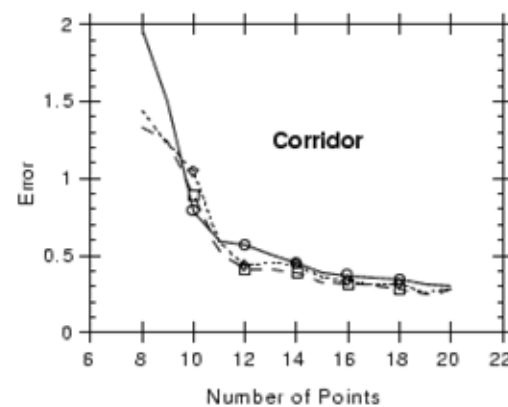
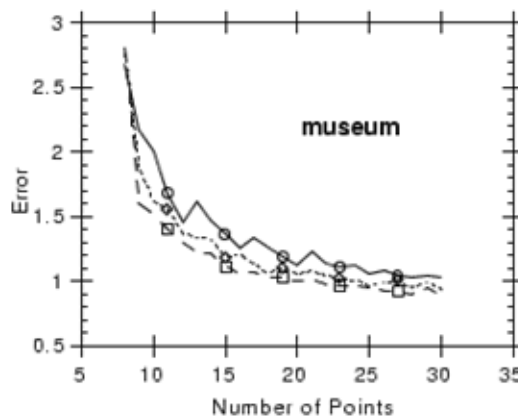
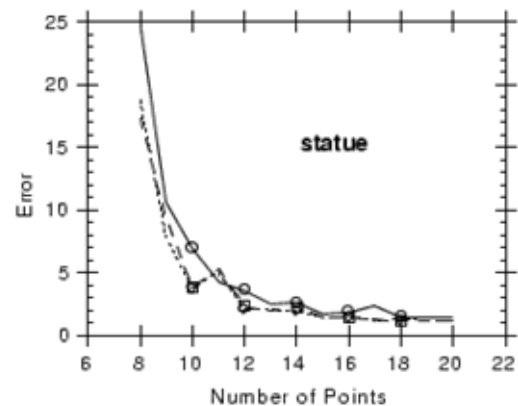
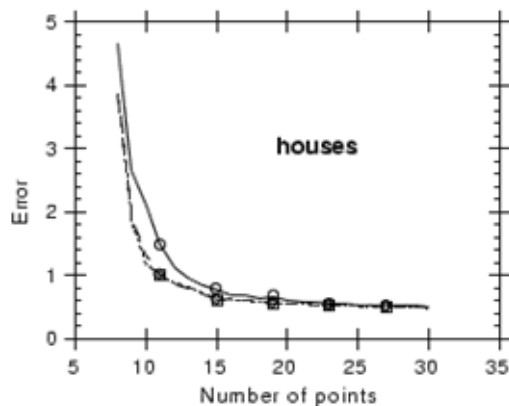
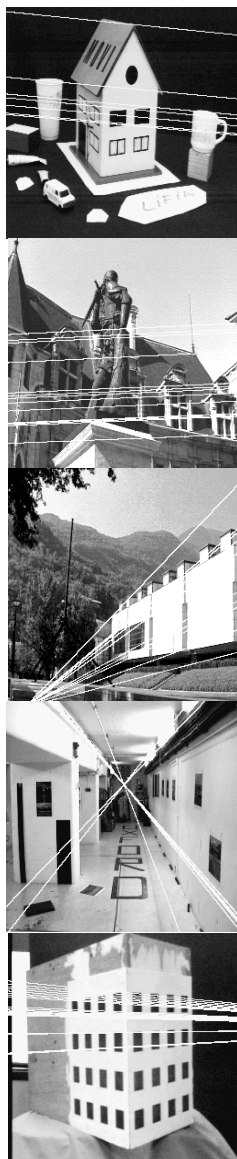
Some Experiments



Some Experiments



Some Experiments



Residual error:

$$\sum_i d(\mathbf{x}'_i, F\mathbf{x}_i)^2 + d(\mathbf{x}_i, F^T \mathbf{x}'_i)^2$$

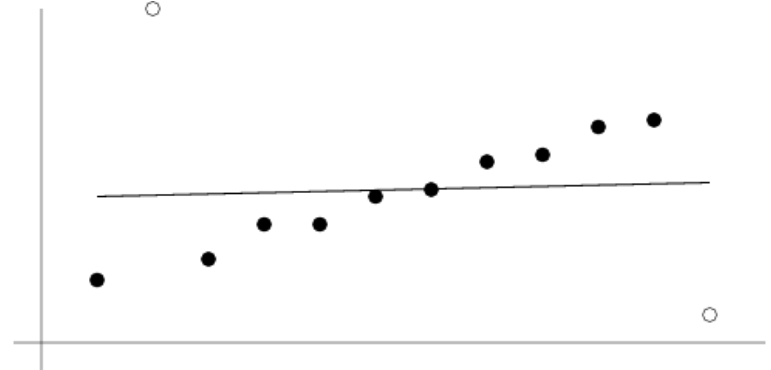
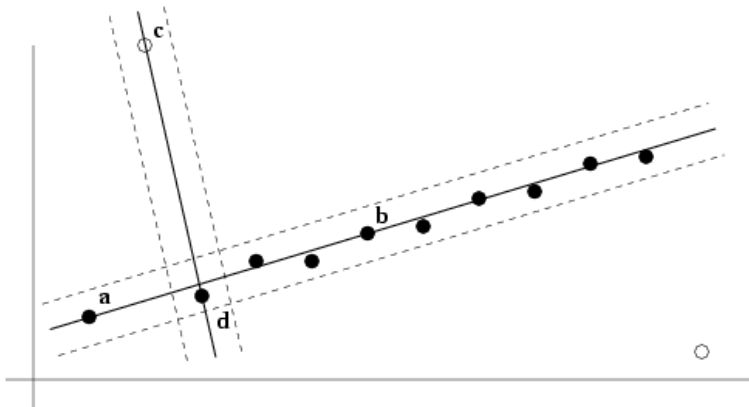
(for all points!)

Recommendations:

1. Do not use unnormalized algorithms
2. Quick and easy to implement: 8-point normalized
3. Better: enforce rank-2 constraint during minimization
4. Best: Maximum Likelihood Estimation
(minimal parameterization, sparse implementation)

Robust Estimation

- What if set of matches contains gross outliers?



RANSAC for F Estimation

Step 1. Extract features

Step 2. Compute a set of potential matches

Step 3. do

Step 3.1 select minimal sample (i.e. 7 matches) } (generate hypothesis)
Step 3.2 compute solution(s) for F }
Step 3.3 determine inliers (verify hypothesis)

until $p(\#inliers, \#samples) > 95\%$ or 99%

Step 4. Compute F based on all inliers

Step 5. Look for additional matches

Step 6. Refine F based on all correct matches

$$p = 1 - \left(1 - \left(\frac{\#inliers}{\#matches}\right)^7\right)^{\#samples}$$

#inliers	90%	80%	70%	60%	50%
#samples	5	13	35	106	382

Finding more matches



restrict search range to neighborhood of epipolar line
(± 1.5 pixels)
relax disparity restriction (along epipolar line)

Degenerate Cases

- Degenerate cases
 - Planar scene
 - Pure rotation
- No unique solution
 - Remaining DOF filled by noise
 - Use simpler model (e.g. homography)
- Model selection (Torr et al., ICCV'98, Kanatani, Akaike)
 - Compare H and F according to expected residual error (compensate for model complexity)

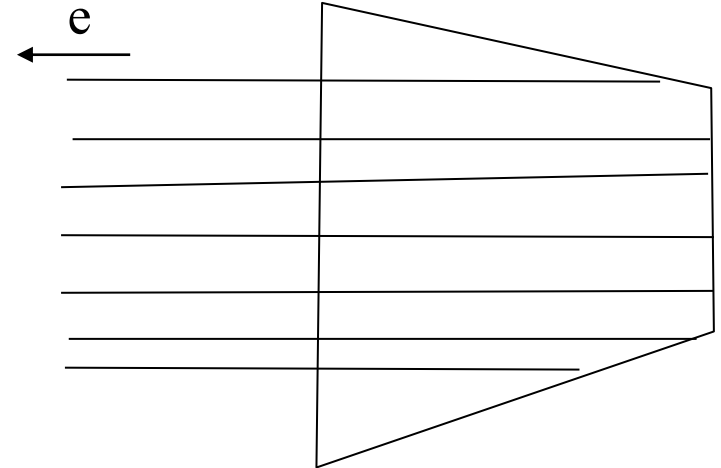
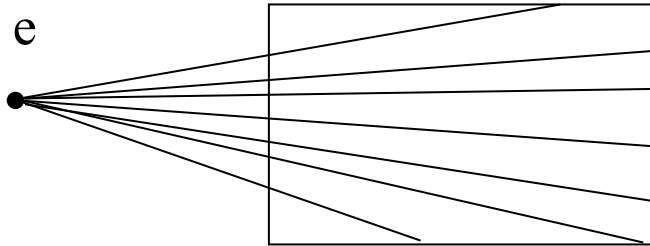
Benefits from having F

- Given a pixel in one image, the corresponding pixel has to lie on epipolar line
- Search space reduced from 2-D to 1-D

Image Pair Rectification

simplify stereo matching
by warping the images

Apply projective transformation so that epipolar lines
correspond to horizontal scanlines



$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = H e$$

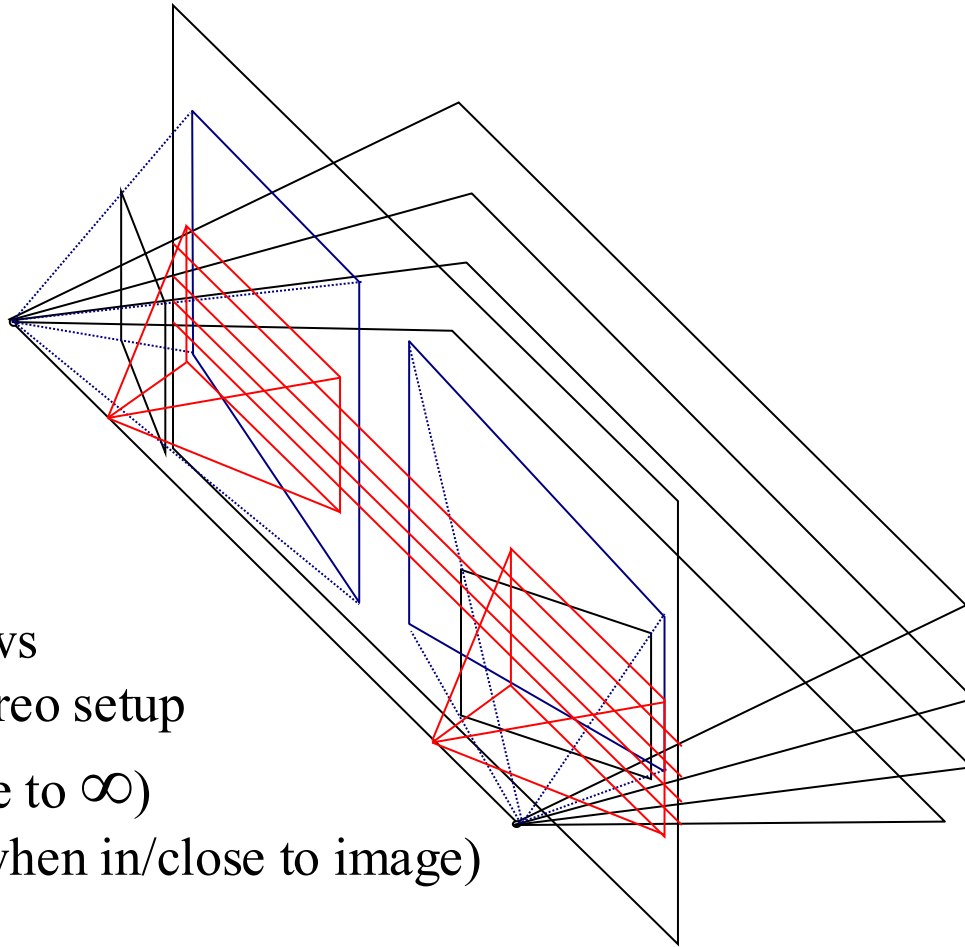
map epipole e to $(1,0,0)$

try to minimize image distortion

problem when epipole in (or close to) the image

Planar Rectification

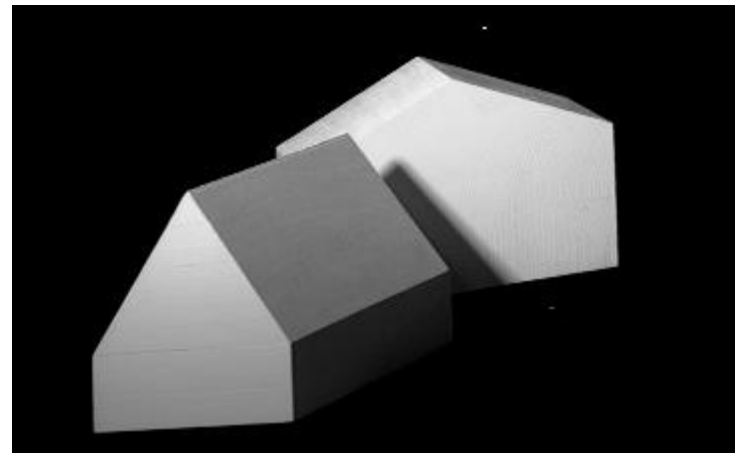
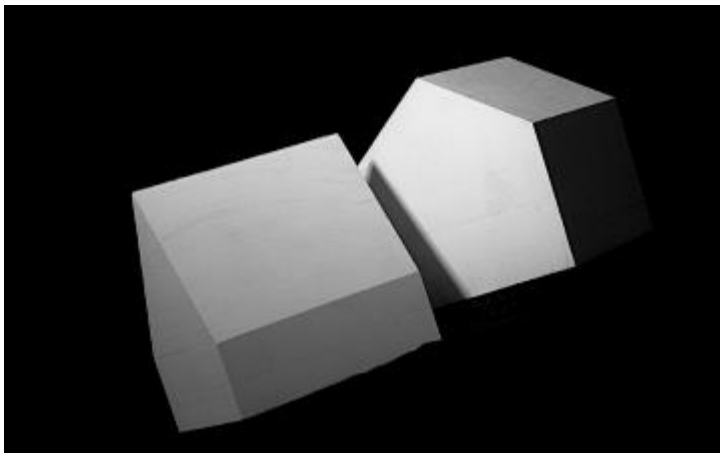
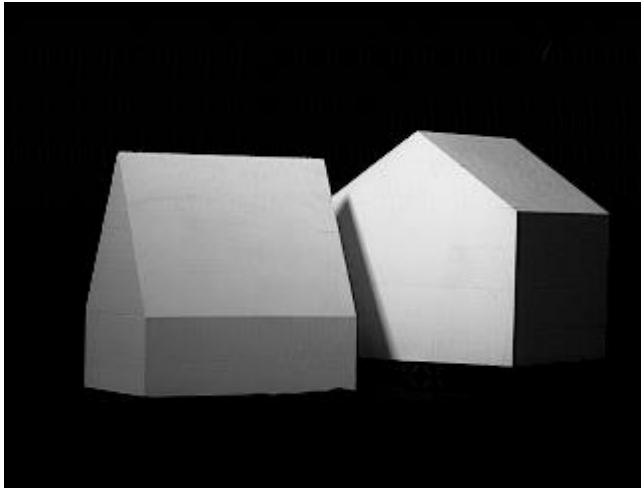
(standard approach)



Bring two views
to standard stereo setup

(moves epipole to ∞)

(not possible when in/close to image)

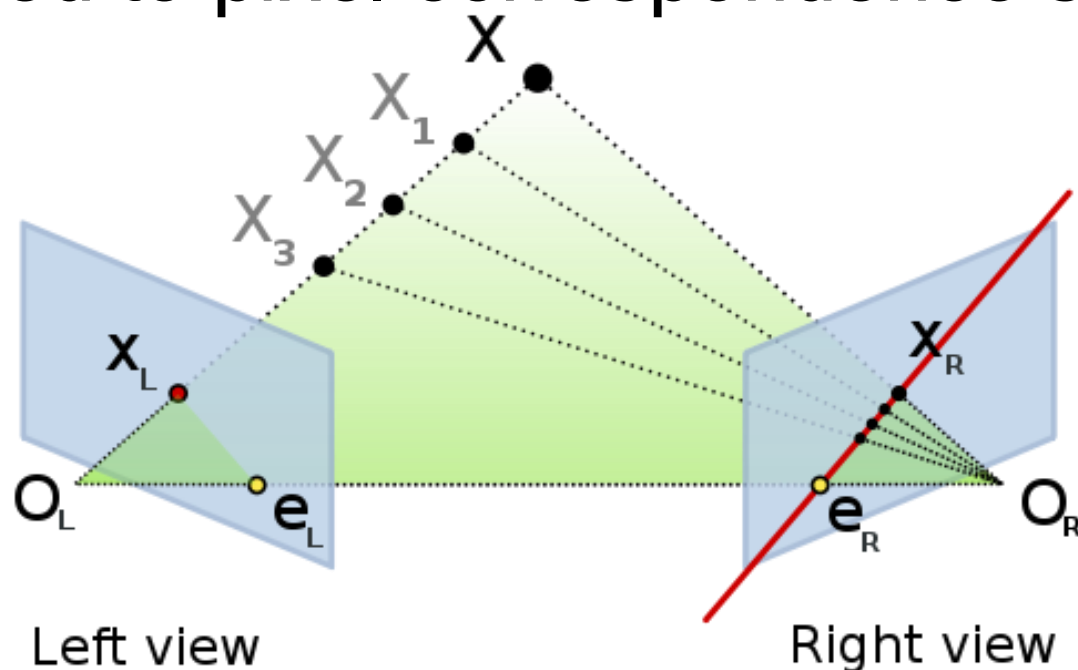


Stereo Matching

Slides by Rick Szeliski, Pascal
Fua, P. Mordohai, E. Dunn

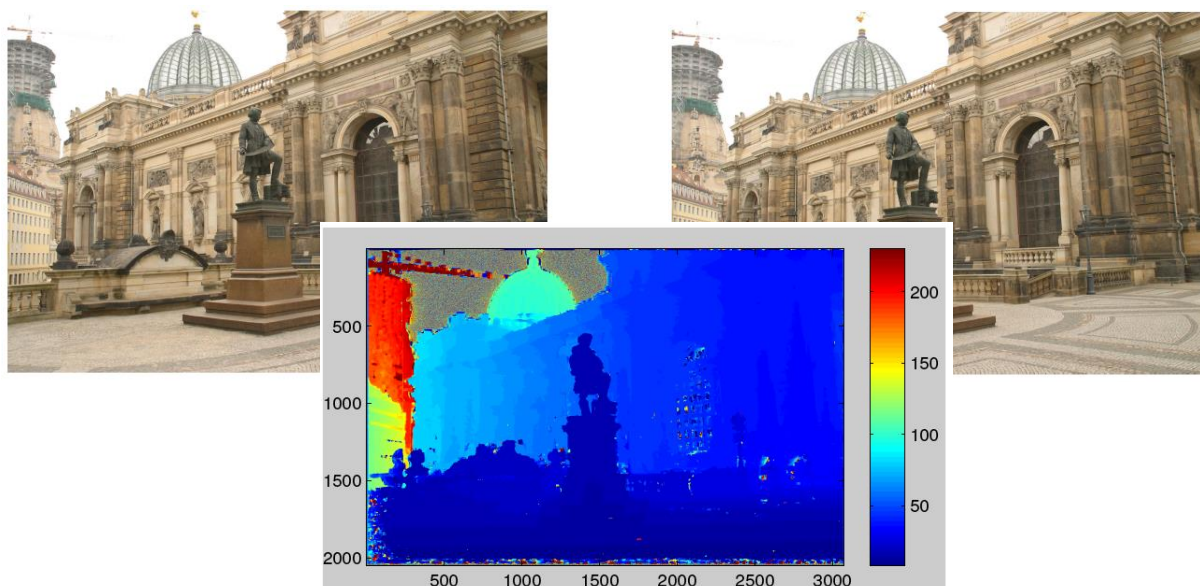
Depthmap Estimation

- Basic Insight: Given a known imaging configuration, geometry estimation is reduced to pixel correspondence search.



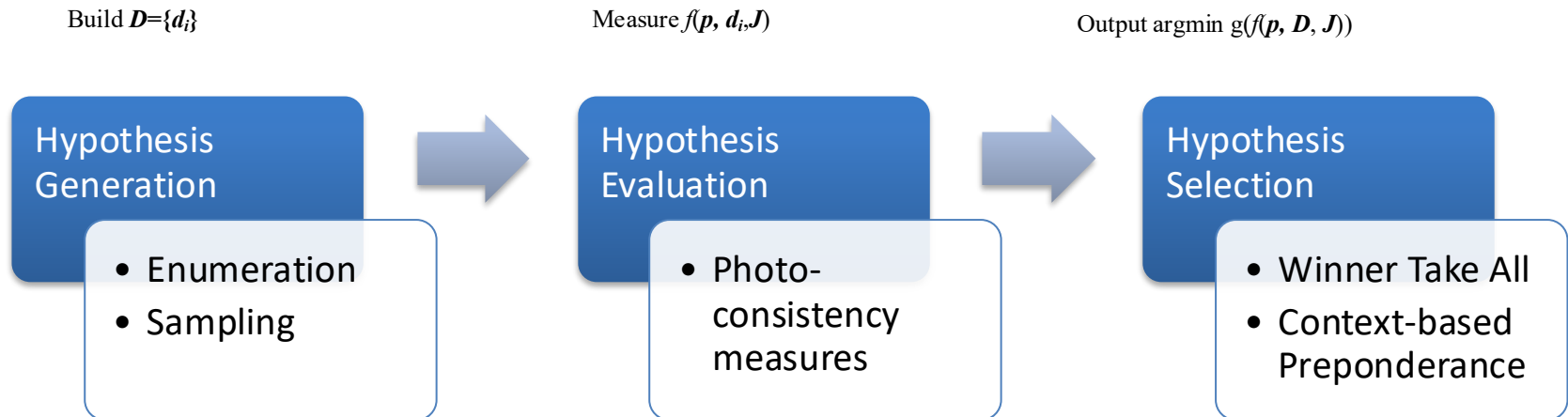
Depthmap Estimation

- Goal: Determine a view-dependent depthfield (*i.e.* pixel-wise estimates) by leveraging the photo-consistency of a set of overlapping images.



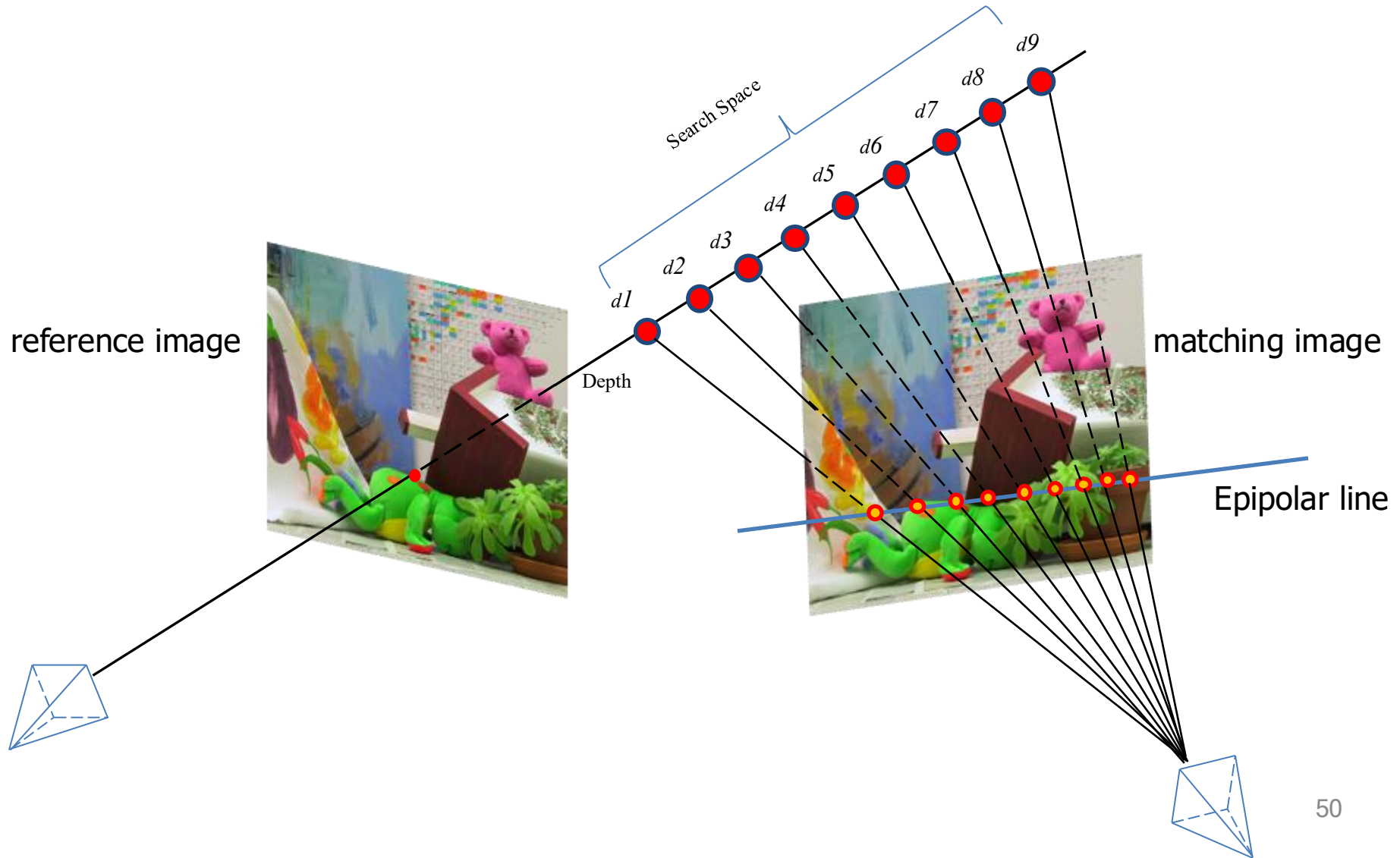
Depthmap Estimation

- For each pixel p in image I determine a depth value, among a set D of depth hypotheses, most congruent with a set J of source images.

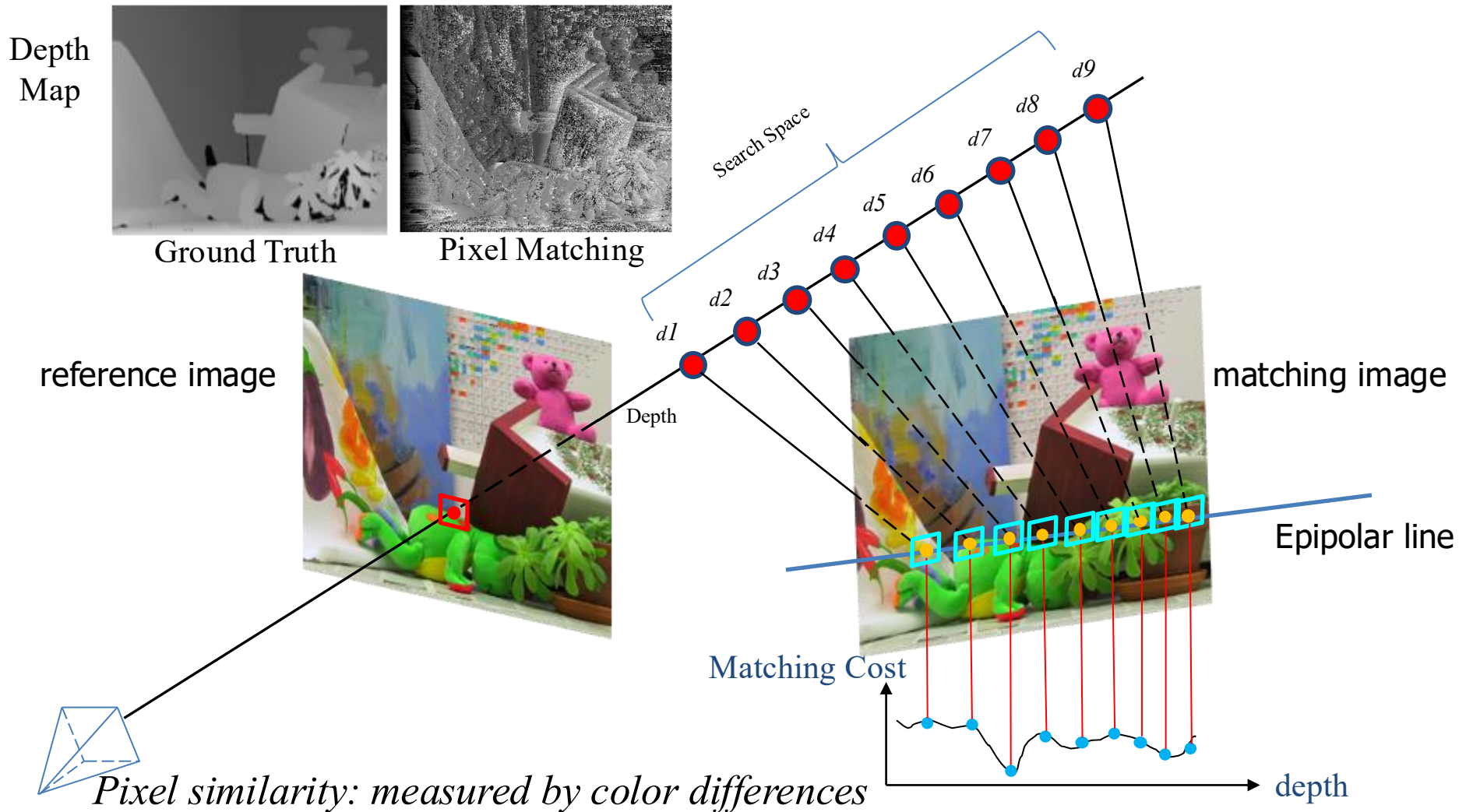


- The two-view case is illustrative ...

Two-View Hypothesis Generation



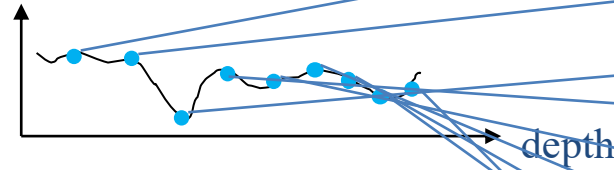
Two-View Hypothesis Generation



Two-View Hypothesis Generation



Matching Cost



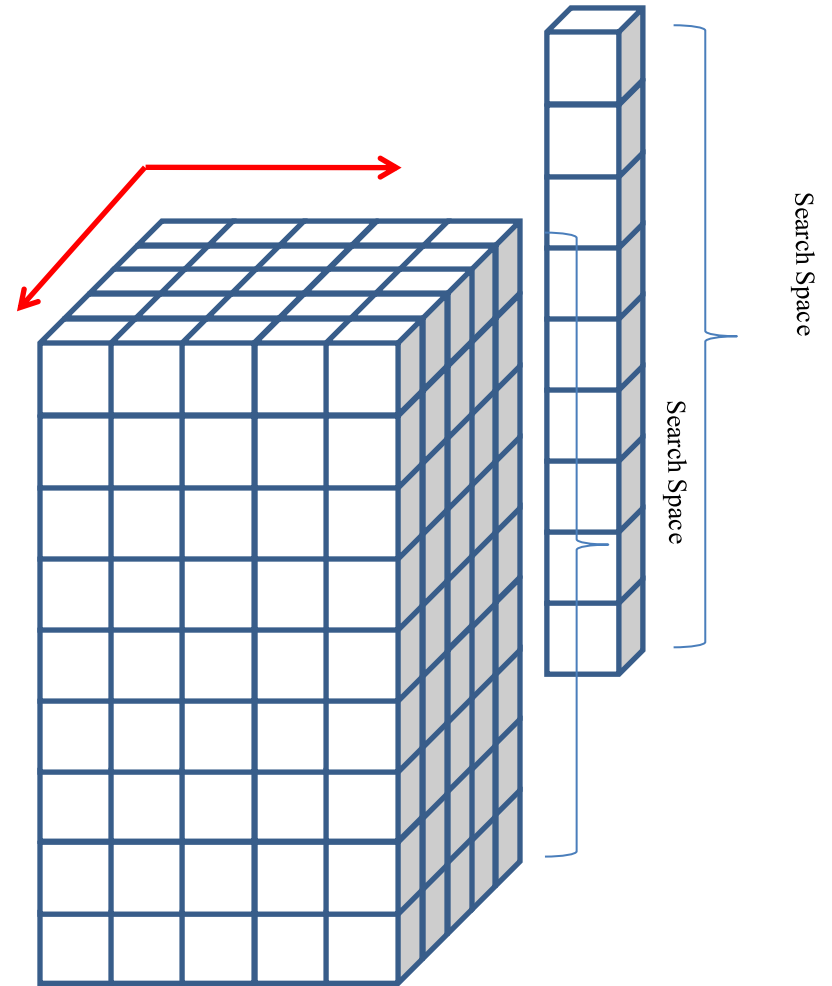
Pixelwise Costs

Search Space

Two-View Hypothesis Evaluation



*Cost aggregation:
cutting the cost volume.*

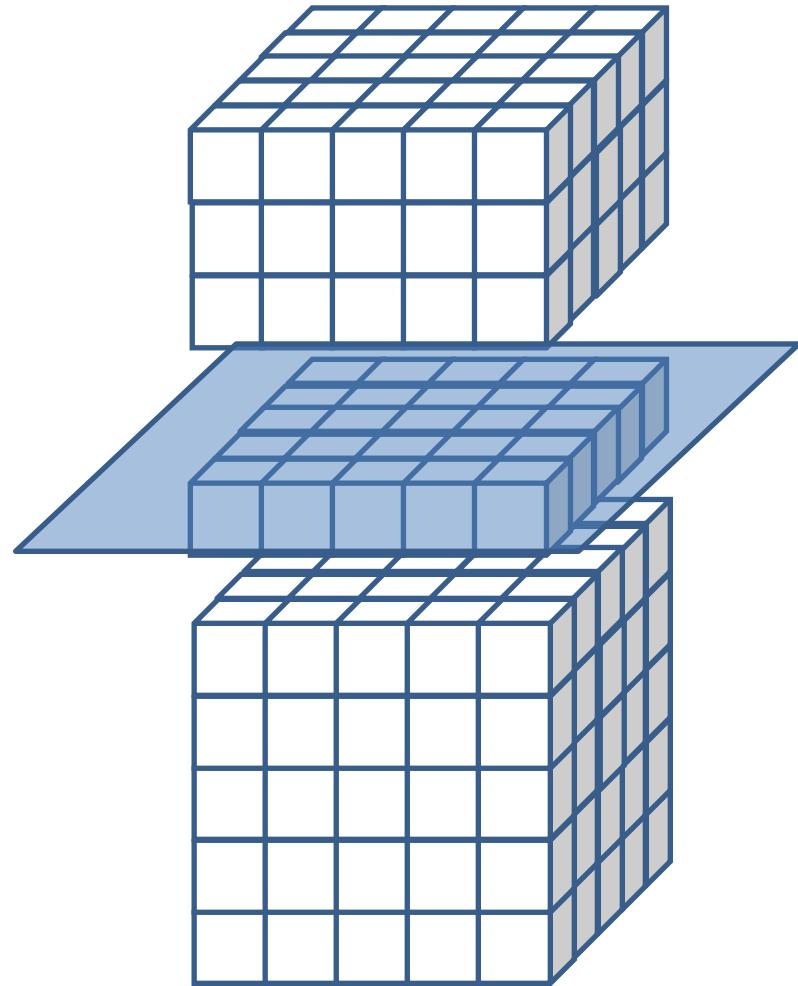


Cost Volume

Two-View Hypothesis Evaluation



Fronto-Parallel Plane



Cost Volume

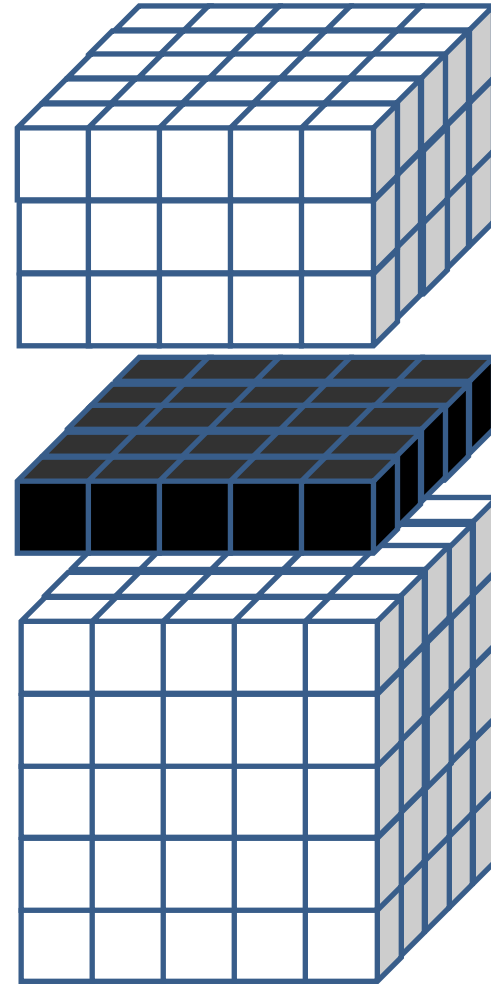
Two-View Hypothesis Evaluation



Treat neighbors equally

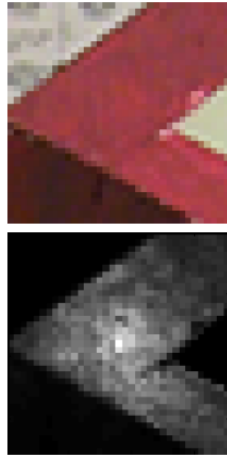


Depth Map



Cost Volume

Two-View Hypothesis Evaluation

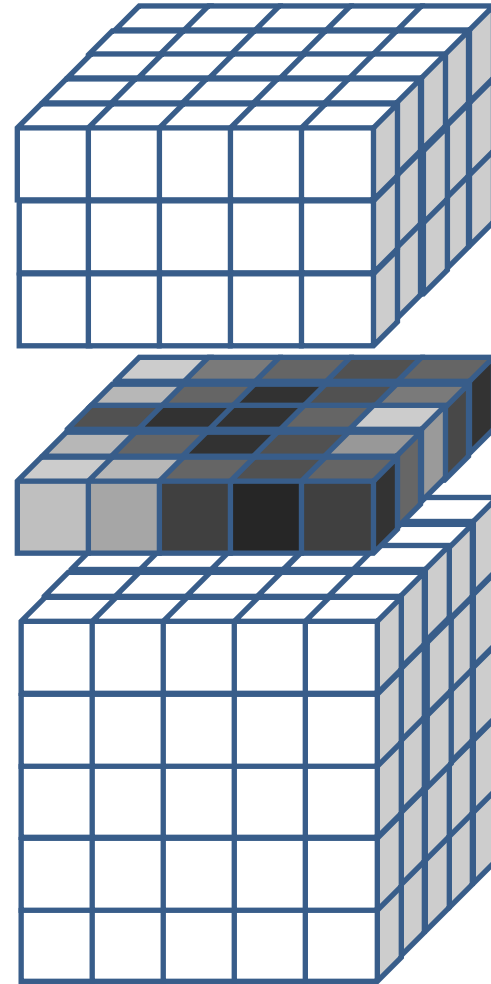


- Color differences
- Spatial distances

Adaptive Weight
Yoon and Kweon, PAMI 2006

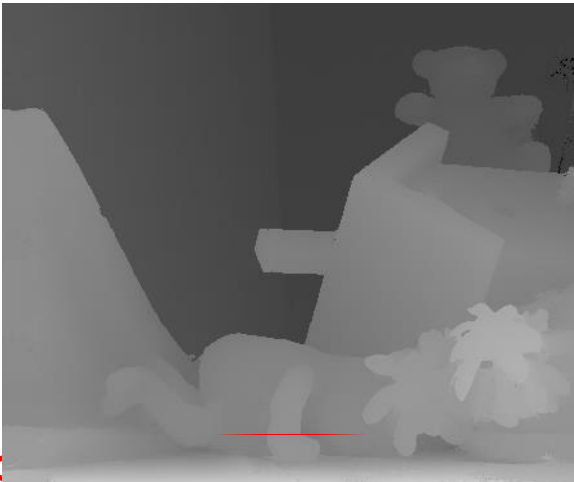


Depth Map



Cost Volume

Two-View Hypothesis Evaluation



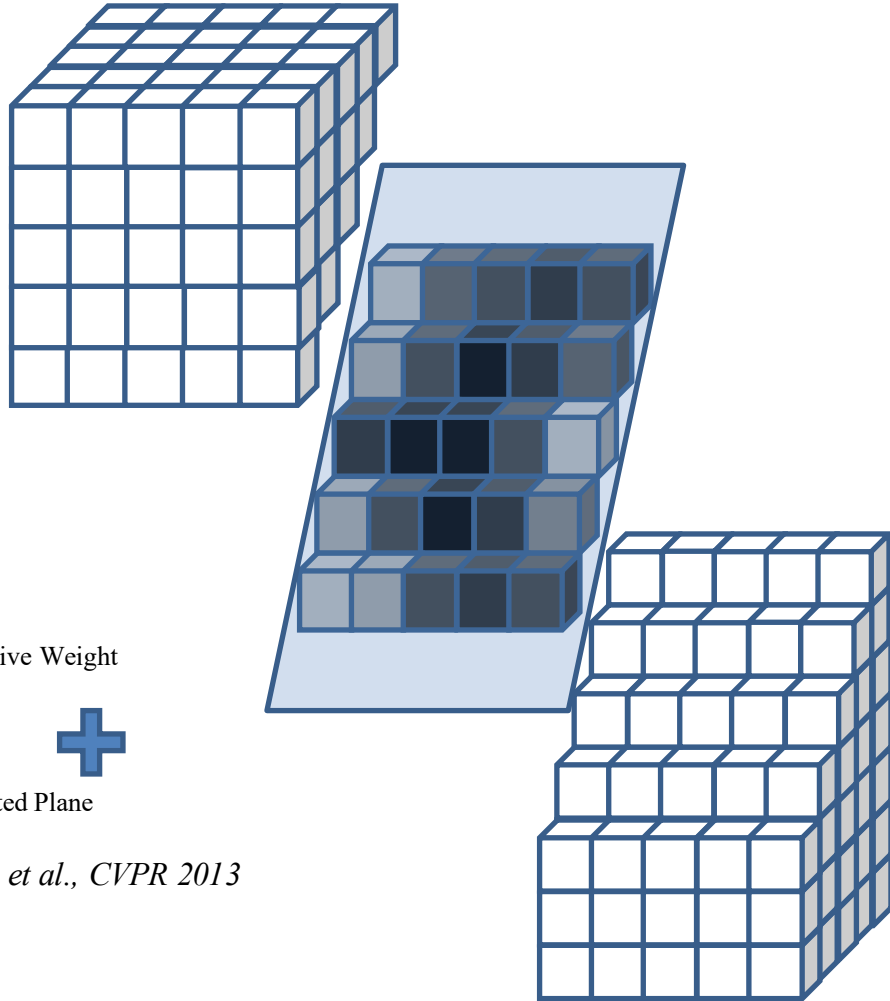
Depth Map

Adaptive Weight



Oriented Plane

Lu et al., CVPR 2013



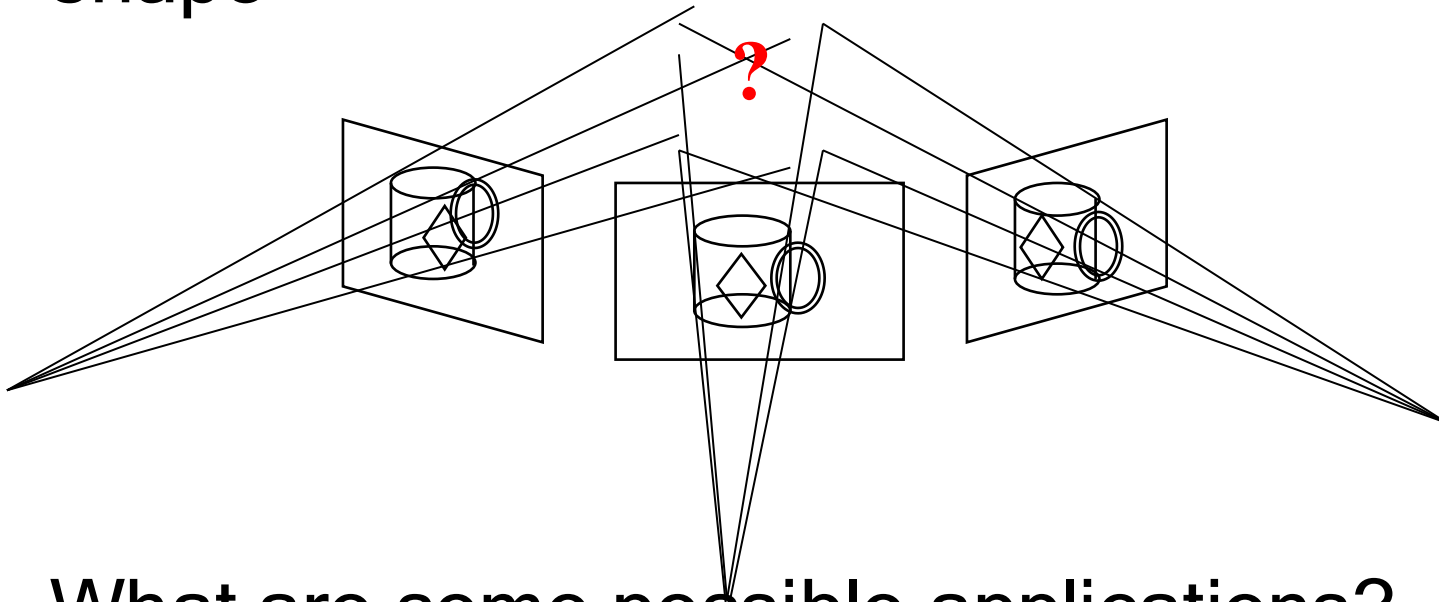
Cost Volume

Aggregated Photo-consistency

- Sum of Squared Differences (SSD)
- Normalized Cross Correlation (NCC)
- CENSUS Transform
- Rank Transform
- Mutual Information
- Learning-based Methods

Stereo Matching

- Given two or more images of the same scene or object, compute a representation of its shape



- What are some possible applications?

Stereo Matching

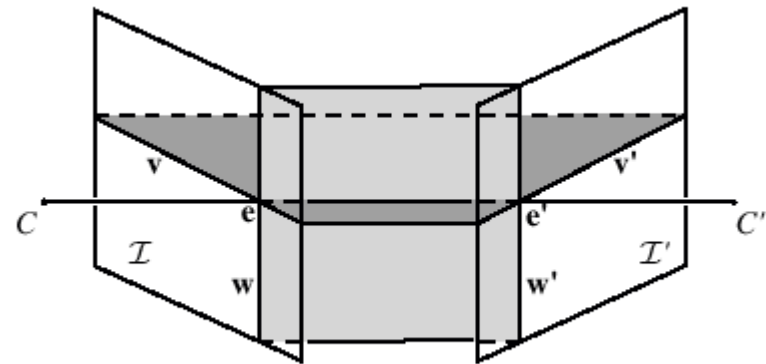
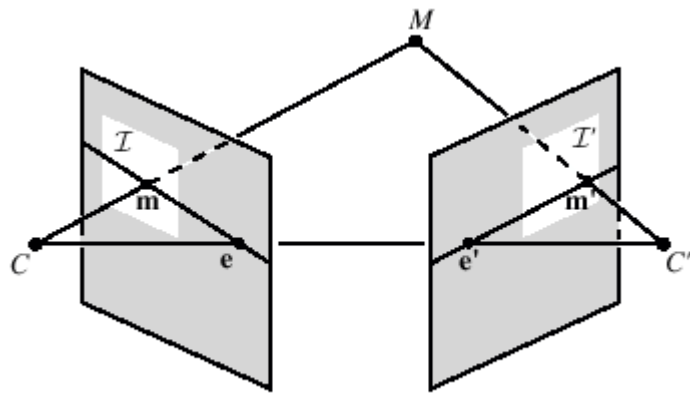
- Given two or more images of the same scene or object, compute a representation of its shape
- What are some possible representations?
 - depth maps
 - volumetric models
 - 3D surface models
 - planar (or offset) layers

Stereo Matching

- What are some possible algorithms?
 - match “features” and interpolate
 - match edges and interpolate
 - match all pixels with windows (coarse-fine)
 - use optimization:
 - iterative updating
 - dynamic programming
 - energy minimization (regularization, stochastic)
 - graph algorithms

Rectification

- Project each image onto same plane, which is parallel to the baseline
- Resample lines (and shear/stretch) to place lines in correspondence, and minimize distortion



- Take rectification for granted in this course

Rectification



(a) Original image pair overlaid with several epipolar lines.



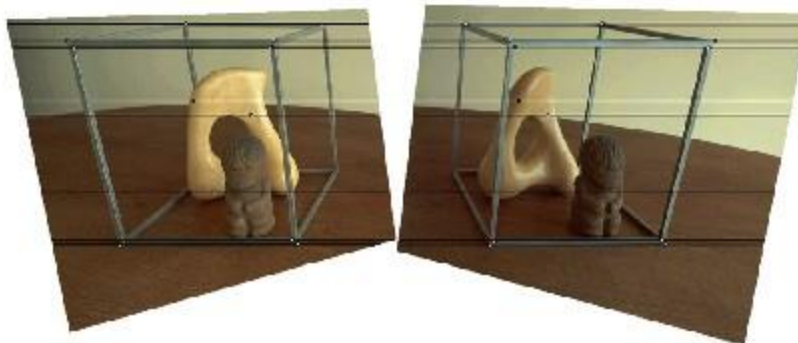
(b) Image pair transformed by the specialized projective mapping $\mathbf{H}_p$ and $\mathbf{H}'_p$. Note that the epipolar lines are now parallel to each other in each image.

BAD!

Rectification



(c) Image pair transformed by the similarity H_r and H'_r . Note that the image pair is now rectified (the epipolar lines are horizontally aligned).



(d) Final image rectification after shearing transform H_s and H'_s . Note that the image pair remains rectified, but the horizontal distortion is reduced.

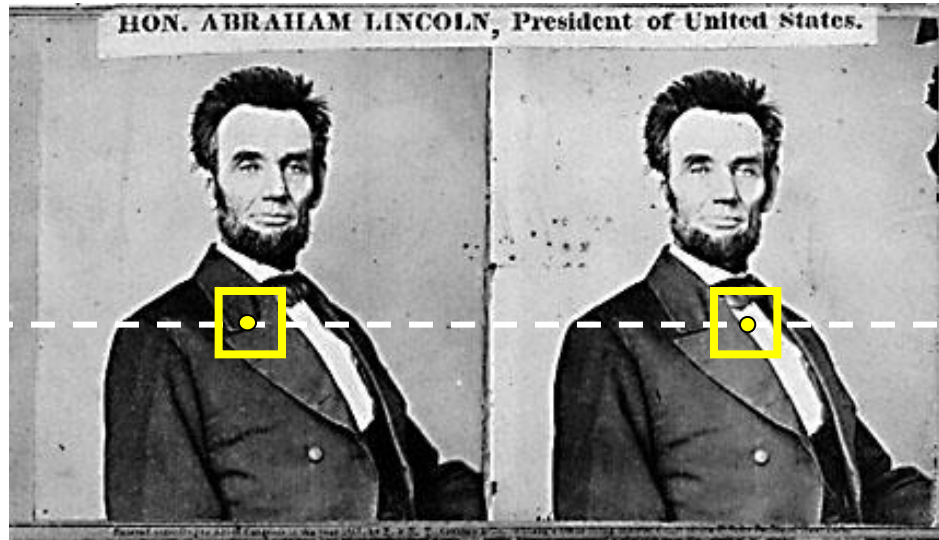
GOOD!

Finding Correspondences

- Apply feature matching criterion at *all* pixels simultaneously
- Search only over epipolar lines (many fewer candidate positions)



Basic Stereo Algorithm



For each epipolar line

For each pixel in the left image

- compare with every pixel on same epipolar line in right image
- pick pixel with minimum match cost

Improvement: match ***windows***

Disparity

- Disparity d is the difference between the x coordinates of corresponding pixels in the left and right image

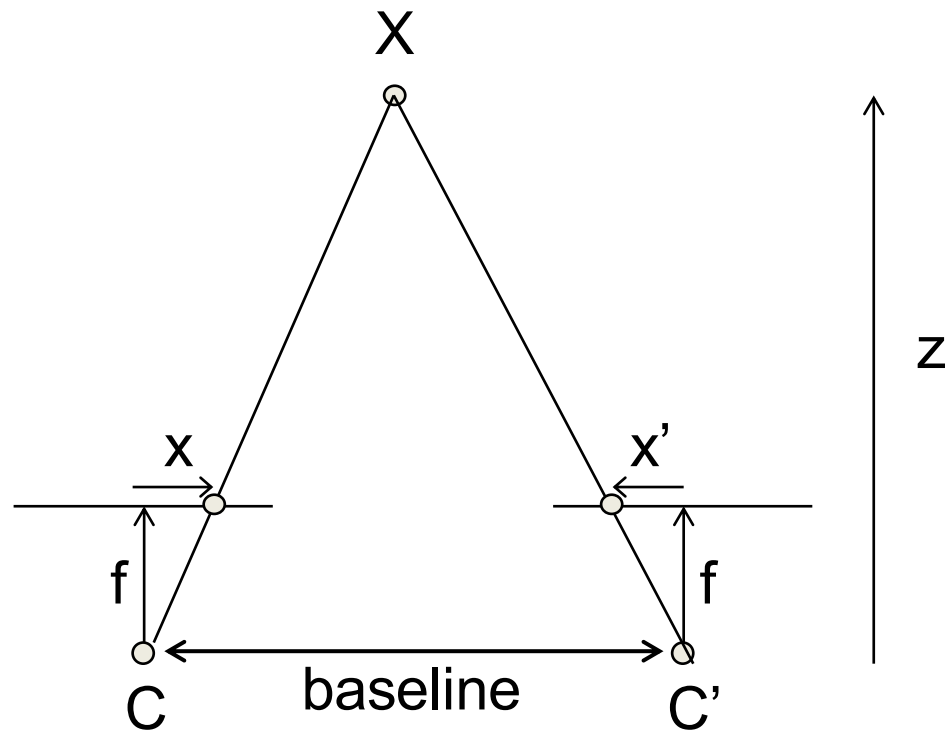
$$d = x_L - x_R$$

- Disparity is inversely proportional to depth

$$Z = \frac{bf}{d}$$

Stereo Reconstruction

$$Z = \frac{bf}{d}$$

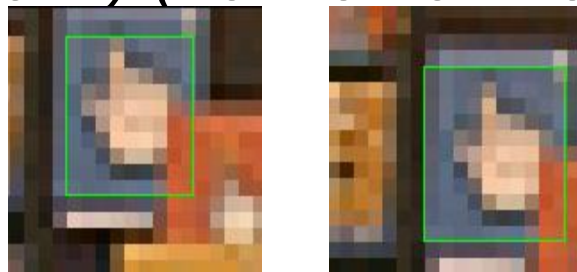


Finding Correspondences

- How do we determine correspondences?
 - *block matching* or *SSD* (sum squared differences)

$$SSD = \sum_{[i,j] \in R} (f(i,j) - g(i,j))^2$$

- d is the *disparity* (horizontal motion)



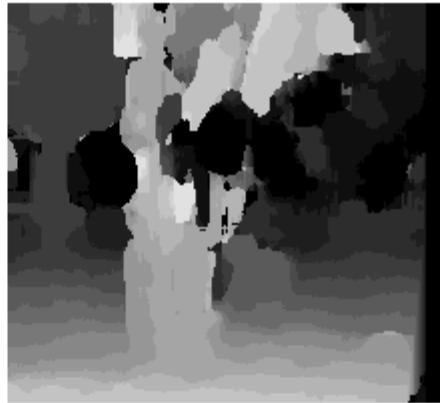
- How big should the neighborhood be?

Neighborhood size

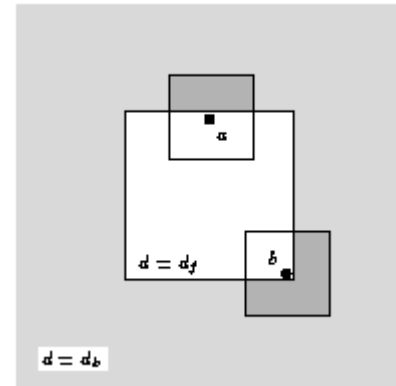
- Smaller neighborhood: more details
- Larger neighborhood: fewer isolated mistakes



$w = 3$



$w = 20$



Challenges

- Ill-posed inverse problem
 - Recover 3-D structure from 2-D information
- Difficulties
 - Uniform regions
 - Half-occluded pixels
 - Repeated patterns



Pixel Dissimilarity

- Sum of Squared Differences of intensities (SSD)

$$SSD = \sum_{[i,j] \in R} (f(i,j) - g(i,j))^2$$

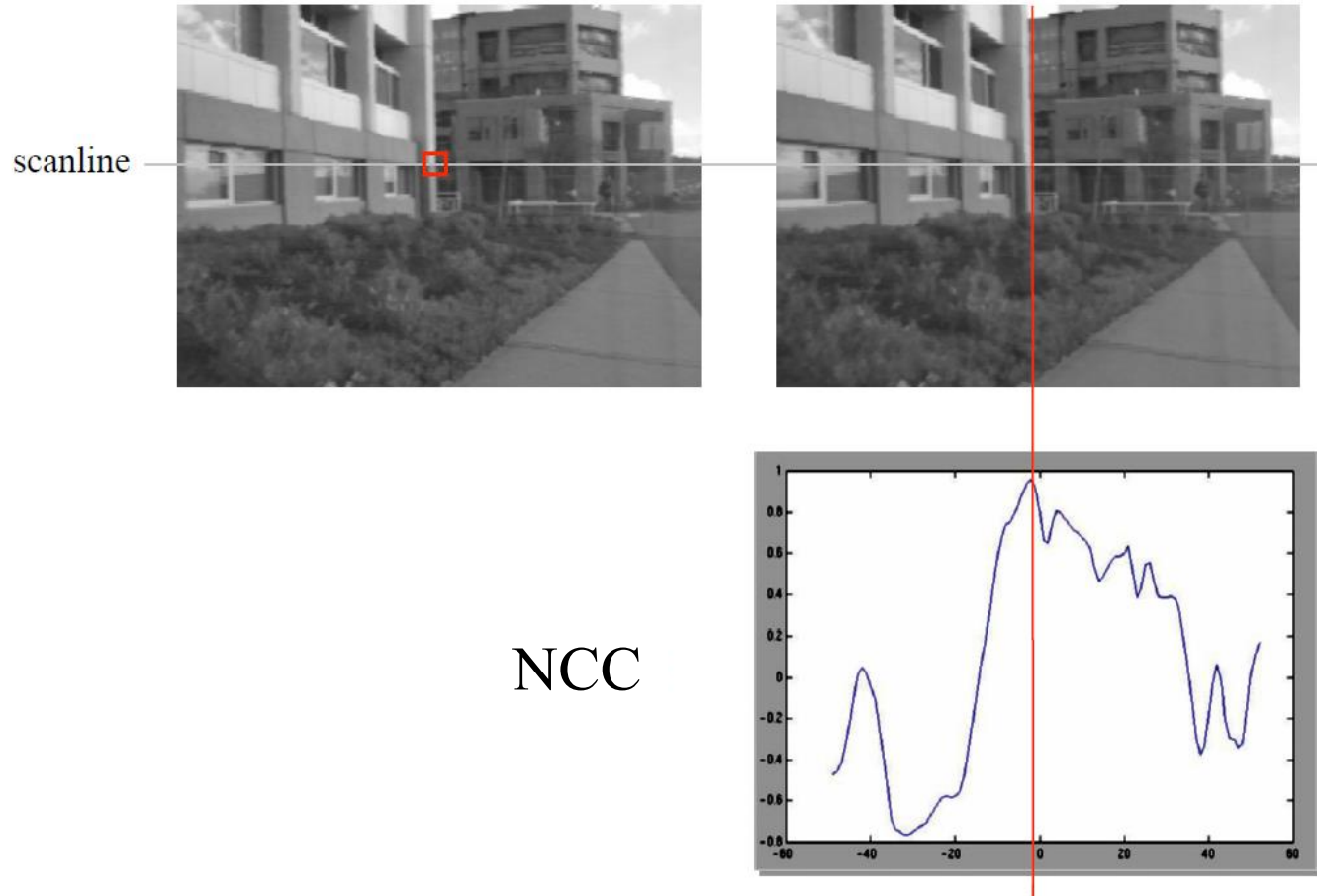
- Sum of Absolute Differences of intensities (SAD)

$$SAD = \sum_{[i,j] \in R} |f(i,j) - g(i,j)|$$

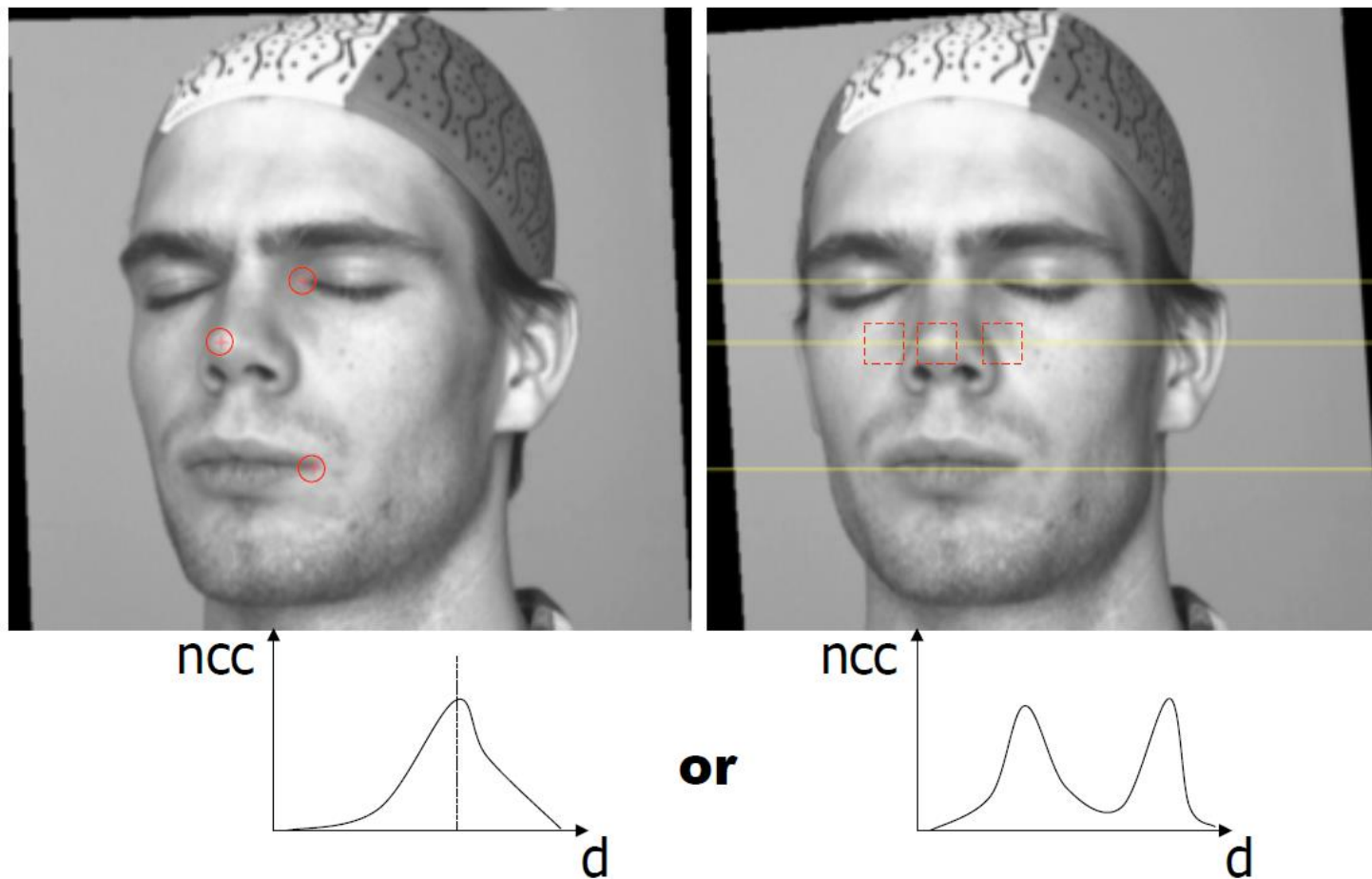
- Zero-mean Normalized Cross-correlation (NCC)

$$NCC(x, y, d) = \frac{\sum_{i \in W} (I_L(x_i, y_i) - \mu_L)(I_R(x_i - d, y_i) - \mu_R)}{\sigma_L \sigma_R}$$

Cost/Score Curve

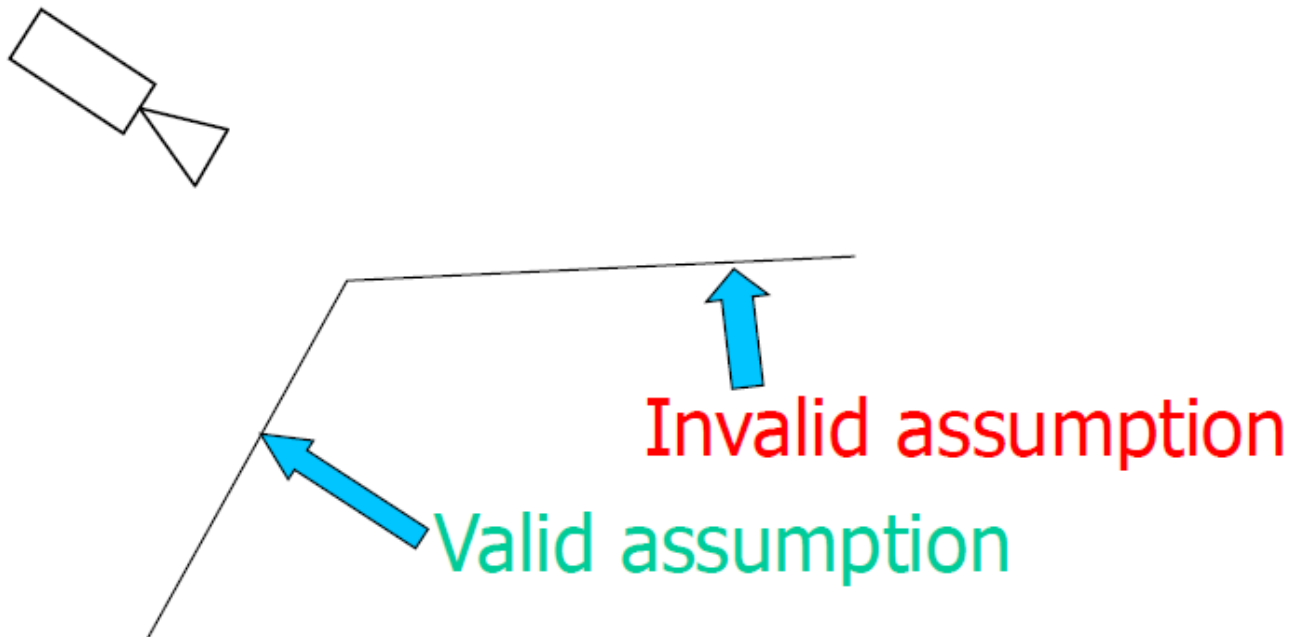


Cost/Score Curve



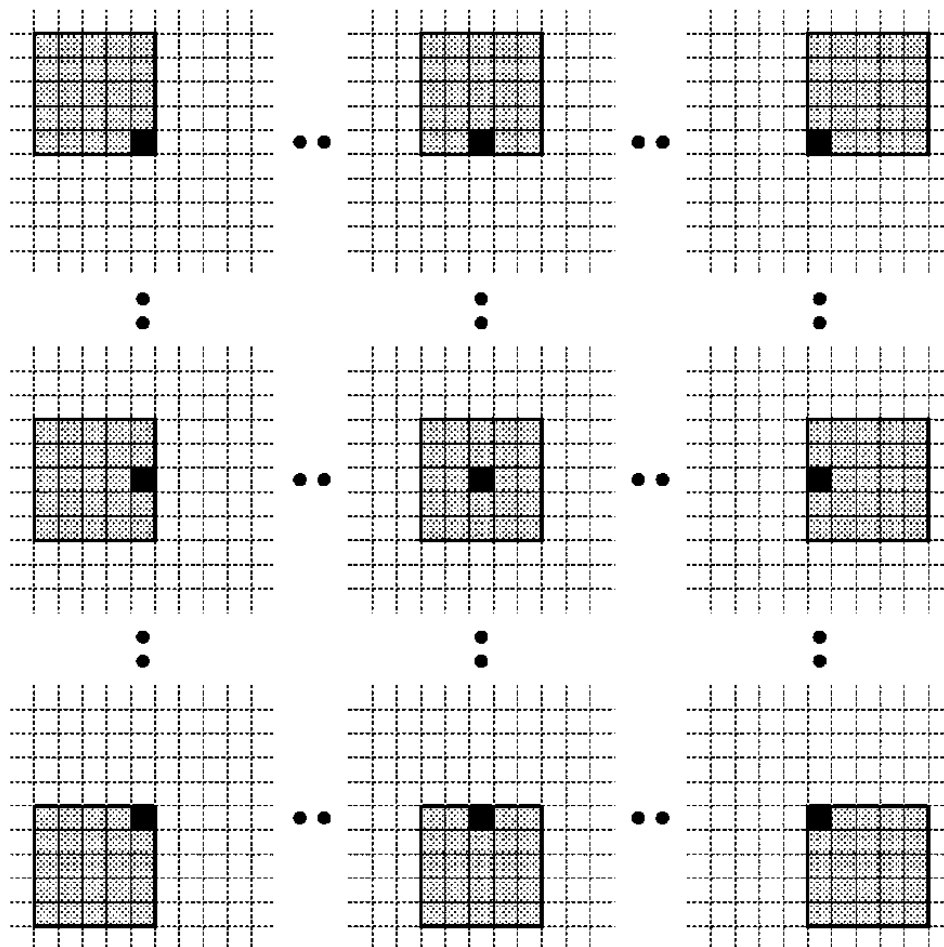
Fronto-Parallel Assumption

- The disparity is assumed to be the same in the entire matching window
 - equivalent to assuming constant depth



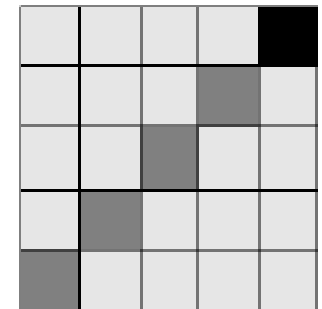
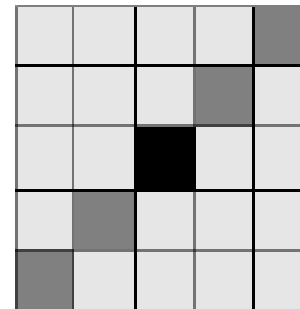
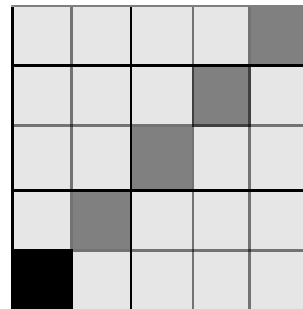
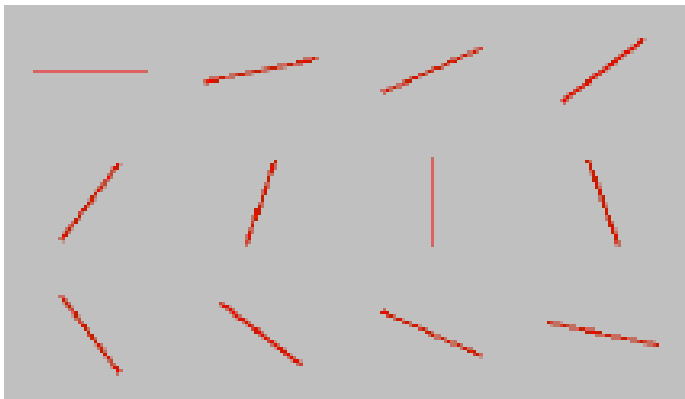
Shiftable Windows

- Avoid having using matching windows that straddle two surfaces
 - Disparity will not be constant for all pixels
- Shift the window around the reference pixel
 - Keep the one with min cost (max NCC)



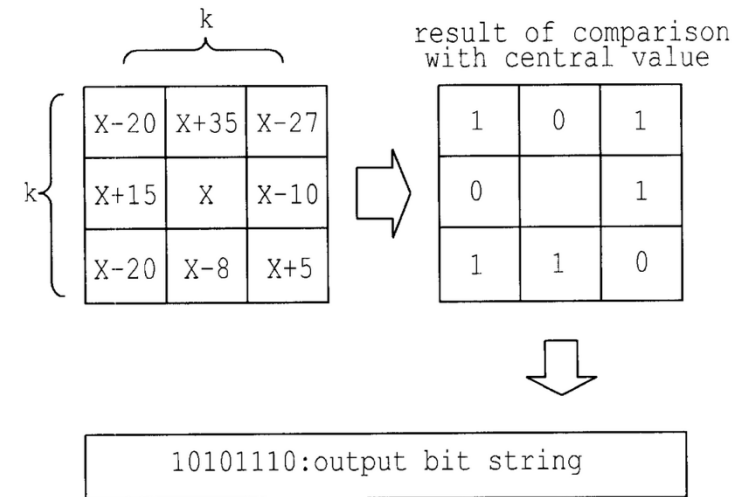
Rod-shaped Filters

- Instead of square windows aggregate cost in rod-shaped shiftable windows
- Search for one that minimizes the cost (assume that it is an iso-disparity curve)



Alternative Dissimilarity Measures

- Rank and Census transforms
- Rank transform:
 - Define window containing R pixels around each pixel
 - Count the number of pixels with lower intensities than center pixel in the window
 - Replace intensity with rank ($0..R-1$)
 - Compute SAD on rank-transformed images
- Census transform:
 - Use bit string, defined by neighbors, instead of scalar rank
- Robust against illumination changes



Locally Adaptive Support

Apply weights to contributions of neighboring pixels according to **similarity** and **proximity**



(a) left support window (b) right support window (c) color difference between (a) and (b)

Locally Adaptive Support

- Similarity in CIE Lab color space:

$$\Delta c_{pq} = \sqrt{(L_p - L_q)^2 + (a_p - a_q)^2 + (b_p - b_q)^2}$$

- Proximity: Euclidean distance
- Weights: $w(p, q) = k \cdot \exp \left(- \left(\frac{\Delta c_{pq}}{\gamma_c} + \frac{\Delta g_{pq}}{\gamma_p} \right) \right)$

Locally Adaptive Support: Results



(a) left image



(b) ground truth



(c) shiftable win. [7]



(d) compact win. [3]



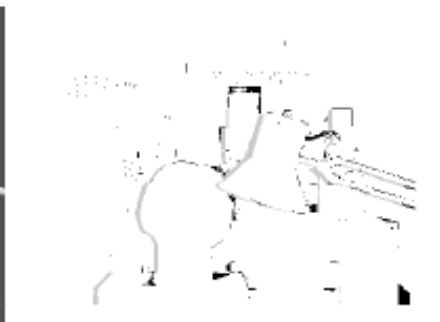
(e) variable win. [4]



(f) Bay. diff. [19]



(g) our result



(h) bad pixels (error > 1)